Differential Performance Fuzzing of SMT Solvers using Genetic Algorithms and LLMs

Adam Jesudason

4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2026

Abstract

Satisfiability Modulo Theories (SMT) describes propositional logic problems, and the software used to solve them, known as SMT solvers, can be used in real-world tasks such as formal verification, test generation, and program synthesis. Despite this, the performance of SMT solvers can be unpredictable because the SMT problem itself is typically NP-hard (and for some theories it is undecidable). As a result, SMT solvers are dependent on complex heuristics in order to achieve good performance in most cases. This lack of consistency can threaten the integrity of the systems that rely on them, including those developed by major technology companies.

This dissertation addresses the problem of differences in solver performance by developing a genetic fuzzing framework that automatically generates SMT instances. It evolves a population of formulas using mutation, crossover, and other operators, biased towards cases where some solvers perform poorly while others succeed (on the same input). This genetic approach is extended by substituting traditional mutation and crossover operations with Large Language Model (LLM)-guided operators, evaluating their effectiveness as alternatives for generating meaningful formula variations.

The results indicate that genetic fuzzing is well-suited to exposing differential solver performance. While LLMs can achieve comparable results, their inability to consistently generate syntactically valid SMT formulas limits their effectiveness at improving fuzzing frameworks. Additionally, the fuzzer exposes the correlation between the comparative root node of a formula and solver-specific performance degradation, along with other inconsistencies of the cvc5 solver.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Adam Jesudason*)

Acknowledgements

I would like to thank Elizabeth Polgreen for her exceptional guidance and supervision throughout the academic year. Equally, I would like to thank her PhD student, Guy Frankel, for his help and expertise inside and outside of standard working hours.

I would also like to thank my parents for their unwavering support throughout my academic journey, which has enabled me to be here and enjoy what have been the best four years of my life.

Table of Contents

1 Introduction **1**
1.1 Usage and Limitations of SMT Solvers . . . 1
1.2 State of the Art Solutions . . . 2
1.2.1 Fuzzing-based approaches . . . 2
1.2.2 LLM-based approaches . . . 2
1.2.3 Gap in Existing Work . . . 3
1.3 Problem Statement . . . 3
1.4 Contributions . . . 3

2 Background **4**
2.1 SMT . . . 4
2.2 SMT Solvers . . . 5
2.3 Fuzzing . . . 6
2.4 SMT Fuzzing . . . 6
2.4.1 BanditFuzz . . . 6
2.5 Genetic Algorithms . . . 8
2.5.1 Mutation . . . 9
2.5.2 Crossover . . . 9
2.6 Distributed Evolutionary Algorithms in Python . . . 10
2.6.1 Types . . . 10
2.6.2 Initialisation . . . 10
2.7 DEAP Implementations . . . 11
2.7.1 Generation . . . 11
2.7.2 Crossover . . . 11
2.7.3 Mutation . . . 12
2.8 Evolutionary Algorithms with LLMs . . . 12

3 Methodology **13**
3.1 Generation of Initial Population . . . 14
3.1.1 Recursive Generation of SMT . . . 14
3.1.2 DEAP Initialisation . . . 15
3.2 Evaluating Fitness . . . 16
3.2.1 Fitness Function . . . 16
3.2.2 SMT Solving . . . 16
3.3 Evolutionary Loop . . . 18

3.3.1 Mutation and Crossover 18
3.3.2 Join 19
3.3.3 Selection 20
3.4 Output 20

4 Use of Large Language Models **22**
4.1 Strategy and Objective 22
4.2 Representation and Format Selection 23
4.3 Prompt Engineering and Constraints 23
4.4 Parsing and Error Handling 23
4.5 Reliability and Success Metrics 24

5 Results **25**
5.1 Experimental Setup 25
5.2 Selection of Baseline Parameters 25
5.3 Performance of Genetic Fuzzing 27
5.3.1 Runtime Difference across Fuzzer Configurations 27
5.3.2 Impact of LLM-Guided Variation Operators 27
5.4 Solver Struggle Distribution 29
5.5 Evolutionary Fitness Progression 30
5.6 Structural Causes of Solver Performance Differences 31
5.6.1 Impact of Formula Depth on Solver Performance 31
5.6.2 Clustering Attempts 32
5.6.3 Formula Scrutinisation 35
5.6.4 Additional Findings 38
5.7 Limitations 38
5.7.1 Large Language Models 38
5.7.2 SMT Solving 39
5.7.3 Data Collection 39

6 Conclusions **40**

Bibliography **41**

A LLM Operator Prompts **43**
A.1 Crossover Prompt 43
A.2 Mutation Prompt 45

B SMT-LIB Formula **47**
B.1 MathSAT Struggle Formula 47
B.2 Declaration Ordering Inconsistency Formula 48

Chapter 1

Introduction

1.1 Usage and Limitations of SMT Solvers

While stemming from one of the simplest forms of pure mathematics known as propositional logic, Satisfiability Modulo Theories (SMT), and particularly their solvers, have a range of real-world applications. This includes program correctness and verification, symbolic execution for systematic and exhaustive test generation, and program synthesis. Beyond this, SMT solvers have also been used for type inference, scheduling problems, and modelling theoretical scenarios [1].

Rungta [2] documents how Amazon Web Services (AWS) scaled a formal-verification tool, which utilised SMT solvers, from generating a few thousand daily SMT solver invocations to a billion daily queries in five years. This was due to the explosive growth of AWS services and the security complexity that came with that. The verification engine which they built and scaled, Zelkova, helps AWS in determining who has access to which resources. This becomes a highly complex problem under various constraints, wildcards, and other conditions that naturally arise in such a company. AWS uses a *portfolio solver* strategy whereby each query invokes multiple SMT solvers and the first result is taken in a "winner-takes-all" system, demonstrating both the depth and breadth of SMT solver usage in the wider industry.

Despite this, SMT solvers still face numerous problems that threaten the integrity of their use cases. Rungta notes that one of the main challenges encountered at AWS was the "lack of monotonicity and stability in runtimes within a given solver version and across different versions". Put simply, this says that there is no real correlation between the size of a query and its runtime, and that performance can vary unpredictably within and across solver versions. SMT solving is not deterministic across all formulas, and some are even undecidable, making it difficult to trust and reproduce solver results. Rungta also outlines the issue of instability in SMT solvers whereby small, non-semantic changes to an input, such as renaming a variable, rerunning identical queries and behaviour across solvers, can all result in inconsistencies which are difficult to explain.

Yi Zhou *et al.* tested their instability detection tool, *Mariposa* [3], on 17,043 SMT queries from various program verification projects and found that "the most recent SMT

solver version is unstable on 2.6% of the queries. For individual projects, the unstable ratio can grow to 5.0%". The paper also claims that SMT solver upgrades often decrease their stability, suggesting that the growing real-world usage of SMT solvers does not necessarily correlate with improved reliability.

Therefore, there is a need for systematic methods to generate challenging SMT formulas that expose solver weaknesses and enable better evaluation. In turn, this could provide means for understanding why these formulas cause difficulty, providing insights that allow solver developers to implement bug fixes or optimisations. Ultimately, this would support the use of SMT and their solvers in real-world applications with greater confidence.

1.2 State of the Art Solutions

1.2.1 Fuzzing-based approaches

In order to stress test SMT solvers, fuzz testing techniques have been applied, including *genetic fuzzing* (fuzz testing integrated into a genetic algorithm with a population of individuals, such as SMT formulas). Genetic programming provides a useful framework for storing, evolving, and analysing a population of inputs. Associated with genetic algorithms are operators, such as mutation and crossover, which alter members of the population. There exist frameworks that provide easy access to such configurations, including built-in genetic operators.

BanditFuzz [4] is a reinforcement-learning-based fuzzer which aims to identify effective mutation strategies by dynamically selecting predefined operators based on observed performance. As a huge inspiration for my research, it is discussed in greater detail in Section 2.4.1. While BanditFuzz had great success in creating SMT which exposed performance issues for solvers, it only considered a single solver's performance at a time, meaning that it may have produced formulas that are universally difficult to solve across all solvers. Such formulas give us less insight into solver-specific weaknesses. While BanditFuzz proved that SMT fuzzing is effective, it did not continue to provide details that could help classify these formulas.

1.2.2 LLM-based approaches

The combination of LLMs with SMT has also been explored to useful effect. LLMs have been used to generate SMT formulas to be used for solver fuzzing. Maolin Sun *et al.* [5] explore how LLMs can be fine-tuned to bias them towards generating bug-prone formulas, by presenting them with a dataset of historical bug-triggering formulas.

A paper exploring LLM-based fuzzing [6] discusses two major drawbacks: a) many generated formulas are syntactically invalid, and b) repeated LLM calls impose a high runtime cost. Subsequently, it proposes the use of LLMs to synthesise reusable "term generators", components that generate logical expressions in accordance with a specified grammar, in order to prevent malformed formulas. This was done by prompting the LLM to extract a context-free grammar for the relevant SMT theories from the official

documentation, and then generate code that implements term generators that adhere to this grammar. Here, LLMs are not called repeatedly during fuzzing, and instead the generators are reused to populate structural skeletons, reducing runtime and ensuring validity.

Large Language Models and Evolutionary Algorithms (EAs) have also been combined in novel ways to interesting effect. Xingyu Wu *et al.* [7] explore how LLMs can be used to generate or mutate candidate solutions in an evolutionary search (*LLM-enhanced EA*), as well as how evolutionary methods can be used to optimise prompts, fine-tuning settings, and model structures (*EA-enhanced LLM*). The first of these is of particular interest to my research, and found that LLMs can function as flexible, high-level variation operators in domains where handcrafted operators are hard (such as code and symbolic expressions, providing promise that this could be extended to SMT-LIB). Together, these papers show the growing ability of language models to create new SMT instances and variations of existing ones.

1.2.3 Gap in Existing Work

Despite the vast usage of LLMs and EAs alongside SMT, there exists no work combining all three in the context of fuzzing. It has been shown that genetic fuzzing is an effective method of exposing weak performance, while LLMs have the ability to support various aspects of the evolutionary process, such as initialisation and variation.

1.3 Problem Statement

This dissertation aims to determine whether a genetic fuzzing approach can be used to automatically generate SMT instances that expose differential performance between state-of-the-art SMT solvers, identifying cases where some solvers perform poorly while others succeed. It further investigates whether Large Language Models can replace traditional genetic mutation and crossover operators to produce more effective, structure-preserving variations. Additionally, it seeks to classify structural properties of SMT formulas that correlate with solver-specific performance degradation.

1.4 Contributions

I developed and evaluated a genetic algorithm–based fuzzer for identifying performance issues in Quantifier-Free Non-Linear Arithmetic (`QF_NIA`) SMT solving (Chapter 3). This approach was extended with Large Language Models to assess their effectiveness as mutation and crossover operators for SMT formulas (Chapter 4). The results show that genetic fuzzing is well-suited to this task, while LLMs can act as meaningful variation operators but struggle to consistently produce syntactically valid formulas. Using this framework, I identified structural patterns in formulas that trigger significant performance degradation across cvc5, Z3, and MathSAT solvers. Additionally, I uncovered specific weaknesses in cvc5, including missing rewrite optimisations and sensitivity to variable declaration ordering (Chapter 5).

Chapter 2

Background

2.1 SMT

Satisfiability Modulo Theories (SMT) represents the problem of determining whether a series of mathematical expressions are satisfiable (by assigning ranges of values to variables). It is an extension of the classical Boolean satisfiability problem (SAT) to include types such as integers, strings, and more complex structures such as lists and arrays.

Referencing Kroening and Strichman's definitions of satisfiability [8], a formula is satisfiable if there exists an assignment of its variables under which it evaluates to `TRUE`. SMT therefore refers to the satisfiability problem over a single or a combination of theories.

A single SMT instance represents the problem of deciding the satisfiability of a first-order logic formula. In SAT, this is a combination of binary variables separated by logical operators. For example:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$$

is a syntactically well-formed SAT formula and has a satisfying assignment of $x_1 =$ `TRUE`, $x_2 =$ `FALSE`, $x_3 =$ `TRUE`.

In the case of SMT, these binary variables can be replaced by predicates containing non-binary variables. In this context, a predicate is a "binary-valued function of non-binary variables" [9], and includes linear inequalities, real arithmetic, uninterpreted functions, strings, and many more. These various facets of an SMT predicate fall under categories called *theories*.

Boolean satisfiability (SAT) problems are NP-complete. By extension, the task of solving SMT is typically NP-hard as they increase the complexity, via theories, of a SAT problem. In some cases, they are even undecidable. Identifying and correlating SMT theories to their decidability is a long-standing problem and often utilises SMT

solvers. The decision procedures that result from decidable SMT are often implemented directly in these solvers.

2.2 SMT Solvers

SMT solvers are programs that aim to solve the SMT problem. They leverage the power of their counterpart: SAT solvers. SAT solvers take a series of Boolean expressions as input and output whether there exists a satisfying assignment for the given variables in the expressions.

SMT solvers primarily consist of the SAT backend, but due to SMT containing theories, they are challenged with finding satisfying assignments for a range of other inputs besides Booleans. SMT solver logic classifies values as *concrete* (constant values such as 5) and *symbolic* (the unknowns that the solver is aiming to attribute values to). For example,

$$3x + 2y = 7$$

is a valid, very simple SMT formula containing concrete values $(1, 2, 3)$ and symbolic variables (x, y) [10].

In general, the work of an SMT solver can be subdivided into two key parts through *theory decomposition*: SAT solving on the Boolean skeleton, and theory-specific constraint checking. To begin this process, each atomic predicate is treated as a Boolean variable. For example,

$$(x > 0 \wedge y < 5) \vee (3x + 2y = 7)$$

can be simplified to

$$(A \wedge B) \vee C$$

by replacing each theory atom (a non-Boolean component) with an arbitrary Boolean variable. In this format, the simplified expression can be passed to a SAT solver, abstracting the underlying meaning of A, B, and C. This is simple combinatorial reasoning and can be solved incredibly quickly.

If the SAT solver finds a satisfying assignment for the Boolean variables, it is then necessary to check the feasibility of the original theory atoms. The theory solver, which in this example is an arithmetic solver, attempts to find concrete values for x and y that satisfy all assigned constraints. This either results in a consistent solution – in which case the SMT solver can output `sat` (satisfiable) along with the corresponding model – or an inconsistent solution. In the latter case, the SAT solver will backtrack and search for another Boolean assignment for the theory solver to validate. Once all satisfying permutations have been exhausted and no consistent solution has been found,

the solver outputs `unsat`. This process is most commonly implemented using the DPLL framework, which underlines solvers like Z3 [11] and cvc5 [12].

2.3 Fuzzing

Fuzzing, or fuzz testing, is an automated software testing technique that provides programs with random data with the aim of exposing unexpected behaviours. This data is repeatedly modified, or fuzzed, in the process of finding vulnerabilities [13]. Typically, fuzzers test programs with structured inputs (making it an attractive approach for SMT solvers) and attempt to provide inputs that are accepted by the parser but contain peculiarities that may expose corner cases. The exact angle and application of fuzz testers will naturally differ according to the software under test and the desired aim of the fuzzer.

The term "fuzz" comes from a 1988 Advanced Operating Systems Class taught by Prof. Barton Miller in which the reliability of UNIX command line programs was tested by executing large numbers of consecutive random inputs until the programs crashed. The name was chosen to evoke the idea of random, unstructured data [14].

2.4 SMT Fuzzing

SMT solvers make for excellent fuzzing candidates for many reasons. Firstly, modern fuzzers, contrary to Miller's original use of the word, are most effective with structured inputs, and SMT solvers boast a complex yet well-structured input language, known as SMT-LIB [15]. Secondly, SMT solvers consist of a variety of different components (SAT solvers, theory solvers, and, in state-of-the-art solvers, optimisation algorithms), creating a naturally problematic environment for bugs to occur and propagate at intersection boundaries.

Despite such wide application, SMT solvers are renowned for their inherent flaws when faced with complex and sometimes, unexplainably, random inputs. Therefore, it is easy to see the motivation for the ongoing improvement efforts, given the wide range of stakeholders that stand to benefit from increased SMT solver reliability.

2.4.1 BanditFuzz

Another state-of-the-art SMT fuzzer is *BanditFuzz* [4], a reinforcement learning-based fuzzer that focuses on identifying grammatical constructs within SMT-LIB that induce performance issues. BanditFuzz reduces the typical problem of how to apply mutations when fuzzing a grammar-based input to a multi-armed bandit problem. Here, arms correspond to possible mutations (varying constructs), and the bandit agents estimate the likelihood of each arm being selected to maximise the objective (runtime slowdown).

BanditFuzz provides the following mutation example for an asserted expression with a maximum depth of two and a fixed number of symbolic variables x_0, x_1:

```
fp.eq(fp.add x0 x1)(fp.sub x0 x1)
```

If the agent elects to insert the construct `fp.abs`, there are two possible results:

```
fp.eq(fp.abs x0)(fp.sub x0 x1),  fp.eq(fp.add x0 x1)(fp.abs x0)
```

For an asserted expression:

```
fp.eq(fp.abs x0)(fp.abs x1)
```

if the agent elects to insert the construct `fp.add`, there are four possible outputs:

```
fp.eq(fp.add x0 x0)(fp.abs x1)
fp.eq(fp.add x0 x1)(fp.abs x1)
fp.eq(fp.abs x0)(fp.add x1 x0)
fp.eq(fp.abs x0)(fp.add x1 x1)
```

The number of permutations is fixed due to type and arity preservation as described earlier (BanditFuzz uses a grammar-preserving mutator). Here, the arms would represent any unary and binary operators that could replace the operators in the original expression while maintaining syntactic validity.

BanditFuzz generates inputs to maximise the relative performance difference between a target solver T and a set of reference solvers R. When a slow instance is found, BanditFuzz ranks syntactic constructs according to their likelihood of contributing to the difference. This ranked output is highly useful for developers compared to a lengthy set of formulas, as it bypasses much of the analysis that would otherwise follow. BanditFuzz brings novel ideas to fuzzing by treating it as a reinforcement learning problem and produces strong results against the target solver Z3. However, the paper is not without flaws and limitations.

BanditFuzz, built on top of *StringFuzz*, a program that randomly generates valid samples from an input grammar, uses a narrow and shallow population of SMT-LIB inputs. Along with providing StringFuzz with the SMT-LIB syntax (in order to generate syntactically correct SMT), BanditFuzz also specifies input generation parameters, documented at the end of Section 4.1 in the paper. It states that they configured StringFuzz to generate "random ASTs at random with five assertions. Each formula has one check-sat call. Each AST has depth three with five string/FP constants" [4]. Following this structure guarantees syntactic validity and makes it easier to identify troublesome grammatical constructs, as the input size cannot drastically change. However, it severely limits population diversity.

This weakness is compounded when analysing the nature of the target and reference solvers used. Although BanditFuzz employs a range of state-of-the-art solvers such as Z3 (and its specific theory solvers), CVC4, MATHSAT, and COLIBRI, it uses Z3 exclusively as the target solver, while the others serve as baseline reference solvers. Thus, despite discovering numerous problematic Z3 inputs, BanditFuzz provides no evidence of the universal strength or transferability of these inputs across other SMT solvers.

The fitness function in genetic algorithms, which is comparable to the reward function in reinforcement learning, is absolutely vital to the success of the model as it defines the optimisation objective. The reward function in BanditFuzz for a well-formed input I is defined as:

$$\text{PerfScore}(I) := \text{runtime}(I,T) - \text{runtime}(I,R)$$

where T and R represent the target and reference solvers, respectively. This score is proportional to the likelihood of selection (the strength of the arm). Though not dissimilar to a performance-based fitness function, $\text{PerfScore}(I)$ is applied as a stand-alone reward for each grammar rule, whereas a genetic fitness function is applied at the input level, quantifying AST structure and combinatorial patterns. This makes it a much richer and more informative objective.

As mentioned, BanditFuzz treats grammar rules as independent "arms" and explores mutations one path at a time by selecting one of these arms in accordance with the reward. The individuals in a genetic programming setup are entire syntax trees (for SMT) that capture combinations of these grammar rules. This allows them to discover non-linear interactions between the basic rules and potentially uncover deeper performance issues.

Ultimately, despite producing impressive results using a unique fuzzing model, BanditFuzz is certainly not a complete solution for SMT performance fuzzing, and it has allowed me to tailor my project toward areas that it does not cover. As mentioned, BanditFuzz employs a singular target solver and therefore is only able to find inconsistencies that relate to Z3 performance. My approach will be able to find and classify performance irregularities across various combinations of solvers in a much more useful time frame. These inconsistencies may occur between any combination of solvers I implement, providing the potential for greater performance deficits to be discovered.

2.5 Genetic Algorithms

Genetic algorithms are a subclass of evolutionary algorithms inspired by the process of natural selection. They feature biologically-named operators such as selection, crossover, and mutation. Featuring a population of candidates, often referred to as *individuals*, they evolve towards better solutions to a predetermined problem [16].

The evolution typically starts from a randomly generated population (the first generation) and proceeds by iteratively evaluating the *fitness* of each individual. Fitness is defined by a problem-specific function, which determines how well an individual performs relative to the desired objective. In my case, the fitness function is a metric about the performance of SMT solvers. As expected through the concept of natural selection, the more "fit" individuals are (stochastically) more likely to be selected for the next generation. This next generation is a mix of existing individuals and newly created ones that arise through the mutation and crossover of individuals from the previous generation.

Genetic algorithms have been used for optimisation problems to find near-optimal solutions when gradient-based methods fail [17], machine learning to select relevant features for classifiers and evolving weights [18], scheduling and planning under constraint [19], and naturally, in bioinformatics problems such as protein structure prediction [20].

2.5.1 Mutation

Mutation is a genetic operator used in evolutionary algorithms to maintain genetic diversity among individuals in a population, analogous to biological mutation [21]. Mutation doesn't have to be a complicated process; flipping a bit in a sequence, known as *single-point mutation*, is a valid method and is representative of a biological point mutation where a single DNA base is altered. Although mutation can cause profound changes to a population over multiple generations, it can be characterised by three requirements [22]:

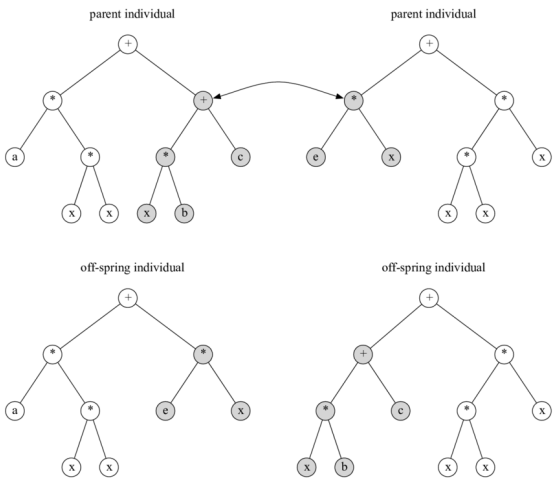
1. Every individual in the population must be reachable by one or more valid mutations.
2. Mutations must not favour particular parts or directions in the potential population space.
3. Small mutations should be more probable than large ones.

These requirements are naturally satisfied in practice, particularly when using specialist genetic algorithm frameworks, which will be discussed shortly.

2.5.2 Crossover

Crossover, also known as *recombination*, is a genetic operator used to combine information from two parent individuals within a population to generate new offspring. Common data structures used for crossover include bit arrays, vectors of real numbers, or trees (which apply to SMT).

When recombining two trees, a *crossover point* (node) is selected in each individual, and the subtrees rooted at those nodes are swapped. This ensures syntactic validity, producing two valid offspring to join the population. However, it does not necessarily guarantee semantic validity. Therefore, additional care must be taken in selecting a crossover point for individuals within a population, depending on their structure and syntax [23].



The example above [24] is a semantically valid crossover because the crossover points (a multiplication and addition node) are both numerical, binary operators with the same domain and thus there are no domain errors, type mismatches or other semantic invalidities.

2.6 Distributed Evolutionary Algorithms in Python

DEAP, a Python framework for evolutionary algorithms, can be used to implement a genetic fuzzer [25]. It provides the necessary data structures and tools to implement techniques such as genetic algorithms, genetic programming, evolution strategies, and many more outside the scope of this project.

2.6.1 Types

DEAP does not force users to choose a predefined type for individuals but instead allows them to build their own. This is done using the `creator` module, and the base setup for an individual can be established in a matter of lines:

```
from deap import base, creator
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", gp.PrimitiveTree, fitness=creator
    .FitnessMax)
```

This creates a fitness class for a maximisation problem (for minimisation we would set `weights=(-1.0)`) as well as an individual class which has the built-in tree type and a fitness attribute set to the fitness function created in the line above. We can see how a fitness function (of sorts) must be defined before we can define our individual. This chronological hierarchy is a recurring theme in DEAP.

2.6.2 Initialisation

As type creation utilised the `creator` module, filling our individuals with values can be done using the `toolbox` module. Following on from the lines above, initialisation is as follows:

```
toolbox = base.Toolbox()
toolbox.register("expr", gp.genFull)
toolbox.register("individual", tools.initRepeat, creator.
    Individual, toolbox.expr)
toolbox.register("population", tools.initRepeat, list, toolbox
    .individual)
```

This creates functions to initialise a population from individuals that are themselves initialised as a full tree (created by `gp.genFull`), the details of which can be specified by adding more arguments. The functions `expr()`, `individual()` and `population()` are thereby registered in the toolbox.

It is now possible to call the function `toolbox.population()` to instantly create a population. This works by repeatedly calling `toolbox.individual()`, which in turn calls `toolbox.expr()`, returning a random tree which is subsequently wrapped into an individual. This iteration ultimately initialises a population, ready for variation, evaluation, and selection.

2.7 DEAP Implementations

2.7.1 Generation

`genFull()`, `genGrow()` and `genHalfAndHalf()` generate trees with all branches to a specified maximum depth, branches grown to a random depth, and a combination of both for structural diversity, respectively.

2.7.2 Crossover

DEAP offers `cxOnePoint()` and `cxOnePointLeafBiased()`, which both work by selecting a random crossover point in each individual and exchanging each subtree with the point as the root between each individual. An example of this can be seen in the diagram in Section 2.5.2. The latter introduces an additional parameter `termpb` which is the probability of choosing a terminal node (leaf) and, as the name suggests, can be used to bias leaf nodes more often, which in turn encourages smaller subtree swaps. When nodes are strongly typed, both functions ensure the second node type corresponds to the first node type to maintain syntactic validity.

In a more complex way, `cxSemantic()` generates a semantic combination of two parents in order to create two children, each with a slight bias towards a parent, defined as:

```
offspring1 = R(x) * parent1(x) + (1 - R(x)) * parent2(x)
offspring2 = R(x) * parent2(x) + (1 - R(x)) * parent1(x)
```

where `R(x)` is a randomly generated expression that outputs values `[0,1]` for each input `x`.

This maintains semantic meaning because it essentially creates a weighted average of both parents determined by the value of the generated expression.

2.7.3 Mutation

DEAP offers a range of mutation functions which alter individuals in different ways.

`mutShrink()` replaces a subtree with a terminal, reducing complexity and size.

`mutUniform()` is the standard and most popular mutation operator in DEAP and simply replaces a randomly selected subtree with a newly generated one (under a generator such as those in Section 2.7.1).

Conversely to `mutShrink()`, `mutInsert()` increases the size of an existing subtree by inserting a new subtree at a randomly chosen point.

Depending on the level of diversity desired in the population, these three main operators can be combined in differing proportions.

2.8 Evolutionary Algorithms with LLMs

As outlined in Section 1.2, Evolutionary Algorithms, Large Language Models, and SMT have been combined to interesting effect on numerous occasions. Outside of my scope, this includes the use of EAs to evolve fuzzer architectures themselves for improved testing performance [26], the use of LLMs to generate and fine-tune bug-prone SMT formulas for solver fuzzing [5], and the creation of term generators via LLMs to ensure syntactic validity and reduce runtime overhead [6]. However, a paper by Xingyu Wu *et al.* [7] describes an approach to utilising LLMs in an evolutionary framework in a general sense, which has particular relevance to my desired implementations.

The paper describes *LLM-enhanced EA* in which LLMs can be used to generate or mutate population candidates in an evolutionary search, as well as define fitness functions. It notes that LLMs can reduce the search space size dramatically. This is done by using LLM-guided initialisation (where the model picks a high fitness starting population) and LLMs to perform evolutionary variation operations, avoiding the stochastic and task-agnostic nature of classic operators (such as the DEAP ones discussed). This was found to vastly speed up convergence (to a suitably strong population) and subsequently lower evaluation cost.

However, Xingyu Wu *et al.* warn that LLM guidance can reduce exploration and diversity, leading to premature convergence. My usage and the dynamic nature of SMT ought to reduce the impact of this potential lack of diversity. They also appreciate the financial and temporal cost of large numbers of LLM calls, particularly in short generational loops. Large Language Models provide famously irreproducible outputs, particularly with detailed prompts. This is certainly notable to my approach due to the complexity that will be required to achieve syntactically valid LLM-varied SMT.

In summary, “Evolutionary Computation in the Era of Large Language Models” documents the ability for LLMs to act as mutation operators in an evolutionary algorithm. I aim to replicate the success demonstrated in this paper and investigate the efficacy of LLMs in fuzzing SMT formulas to find large performance differentials across solvers.

Chapter 3

Methodology

Algorithm 1 provides detailed pseudocode of the entire genetic algorithm developed. The bold comments reference sections which contain more detail on implementation.

Algorithm 1 Genetic Fuzzer Pseudocode

Require: POP_SIZE, NGEN, jnpb, mutpb, cxpb, timeout

$P \leftarrow$ initialise population of size POP_SIZE ▷ **DEAP Generation**
Evaluate fitness of all $ind \in P$ ▷ **Evaluating Fitness**
Remove all $ind \in P$ where ind.flag $=$ `ERROR`
for $gen = 1$ **to** NGEN **do** ▷ **Evolutionary Loop** Start
  $C \leftarrow \text{varAnd}(P,\ cxpb,\ mutpb)$ ▷ **Mutation and Crossover**
  $J \leftarrow \emptyset$
  for each consecutive pair (a,b) in $P \cup C$ **do**
    if rand() $< jnpb$ **then**
      $J \leftarrow J \cup \{\text{join}(a,b)\}$ ▷ **Join**
    end if
  end for
  $O \leftarrow \text{deduplicate}(P \cup C \cup J)$
  Evaluate fitness of all unevaluated $ind \in O$ ▷ **Evaluating Fitness**
  Remove all $ind \in O$ where ind.flag $=$ `ERROR`
  Sort O by fitness (descending)
  $P \leftarrow O[1\,..\,\text{POP_SIZE}]$ ▷ **Selection**
end for ▷ **Evolutionary Loop** End
$ind^* \leftarrow \arg\max_{ind \in P}\ ind.\text{fitness}$ ▷ **Output** Start
if ind^* has a struggle solver $\mathcal{S}$ **then**
  for each solver $s \in \mathcal{S}$ **do**
    $t_{\text{slowest}} \leftarrow \text{evaluate}(ind^*,\ s,\ 10min)$
    $\delta_s \leftarrow (t_{\text{slowest}} - t_{\text{fastest}})\ /\ t_{\text{fastest}}$
  end for
  $RDD \leftarrow \arg\max_{s \in \mathcal{S}}\ \delta_s$ ▷ Relative Runtime Difference Score
end if
Write ind^*, RDD, s to file ▷ **Output** End

Figure 3.1 shows an overview of the genetic fuzzer. The introduction of LLMs, discussed in the next chapter, took place in the *Mutation* and *Crossover* stages of the evolutionary loop.

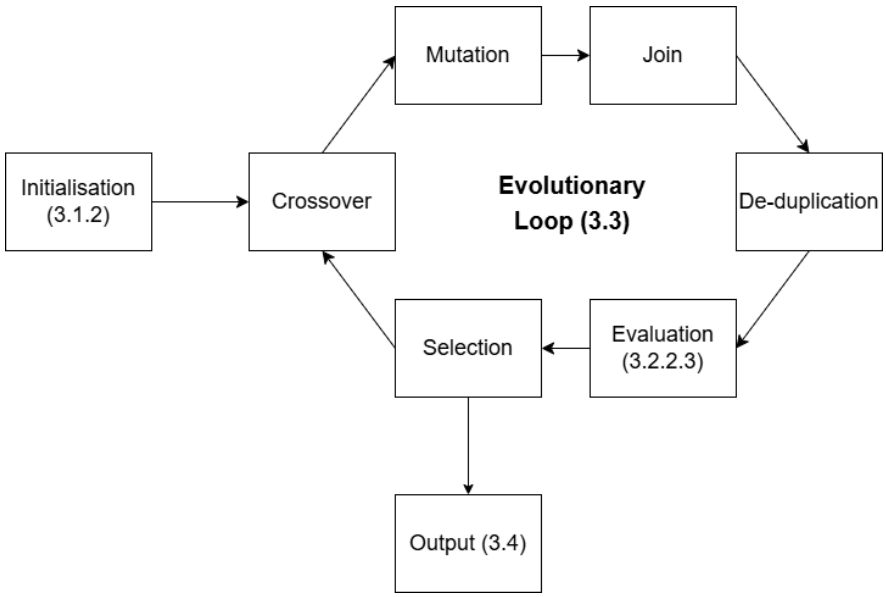


Figure 3.1: Block diagram of the genetic fuzzing pipeline

3.1 Generation of Initial Population

3.1.1 Recursive Generation of SMT

Initially, I used a recursive algorithm that utilised a random number generator to determine the depth of the formula and populated it with random choices of specified operators and values. For example, the depth of the well-formed SMT formula:

$$x_0 + (x_3 * x_2) >= x_1 * (x_0 - x_2)$$

is three. This is because, looking at the operators, our top level at a depth of 1 is the greater-than-or-equal-to, the operators at a depth of 2 are the addition (on the left-hand side) and the multiplication (on the right-hand side), and finally the inner multiplication and subtraction respectively represent a depth of 3. This can be more easily understood by considering the formula in prefix notation, which is widely used as it directly reflects the abstract syntax tree (AST) representation, removing any ambiguity on precedence and allowing for easy parsing.

$$\begin{aligned}(\ >= (+\ x_0\ (*\ x_3\ x_2)) \\ (*\ x_1\ (-\ x_0\ x_2)))\end{aligned}$$

shows the formula in prefix notation, clearly demonstrating the depth of 3.

Although my recursive algorithm was capable of creating valid SMT formulas, to a specified depth, I quickly realised that it was not easily expandable to integrate other theories. In addition, the way in which these formulas were stored was going to cause difficulty when I inevitably wanted to vary them via mutation and crossover. Although I did not end up utilising this recursive algorithm for creating my SMT, it was functional and, more importantly, helped me to realise that my SMT formula had to be created and stored more dynamically than I had first thought.

3.1.2 DEAP Initialisation

I discovered that I could generate and store AST representations of my SMT formula using the DEAP Primitive Tree type [25], as shown in Section 2.6. This allowed me to generate the initial population and apply variation operators, whether built-in (DEAP) or custom made (LLM-guided).

For my use case:

```
pset = gp.PrimitiveSetTyped("MAIN", [ArithRef]*NUM_VARS, BoolRef)
```

specifies that my primitive trees will contain up to `NUM_VARS` arithmetic typed variables with the root operator producing a Boolean value (as required by SMT formulas).

My initial SMT formulas can be represented with the following BNF (where a single SMT formula is a `<BoolExpr>`):

```
<BoolExpr> ::= <ArithExpr> ">"  <ArithExpr>
             | <ArithExpr> "<"  <ArithExpr>
             | <ArithExpr> ">=" <ArithExpr>
             | <ArithExpr> "<=" <ArithExpr>
             | <ArithExpr> "==" <ArithExpr>

<ArithExpr> ::= <ArithExpr> "+" <ArithExpr>
              | <ArithExpr> "-" <ArithExpr>
              | <ArithExpr> "*" <ArithExpr>
              | <Var>

<Var> ::= ARG0 | ARG1 | ... | ARG[NUM_VARS]
```

N.B. This BNF does not represent the exact syntax of a DEAP Primitive Tree, but defines the set of logical formulas that can be generated. The mutation and crossover operators, in DEAP (explicitly) and the LLM-guided versions (through prompting), also abide by this BNF. The only operator that can generate formulas outside of grammar is join, as discussed in Section 3.3.2.

These individuals had a random depth between 1, which would be a formula such as $(ARG0 > ARG1)$, up to MAX_DEPTH (which was varied to see the effect on fuzzer performance). The population size was also a varied parameter.

3.2 Evaluating Fitness

3.2.1 Fitness Function

A fitness value, calculated from a fitness function F, is what defines the evaluation of an individual in an evolutionary algorithm. For my fuzzer, this fitness function was by default the relative difference of the solver runtimes in seconds (*RRD*). This is defined as

$$F(runtimes) = (slowest - fastest) \ / \ fastest$$

where *runtimes* is a list of the Z3, cvc5 and MathSAT solve times, $slowest = max(runtimes)$ and $fastest = min(runtimes)$. $slowest$ in some cases is simply the timeout duration. $fastest$ is the fastest runtime across the other two solvers (or potentially one, if two solvers timed out). However, in the case that the SMT formula was not solved within the timeout constraint by at least one of the solvers, an additional reward is given to that individual. In this case,

$$F(runtimes) = 1000 + (timeout \ / \ fastest)$$

where the $timeout$ is a parameter designed to encourage the survival of formulas that time out, rather than those with large runtime differences between solvers but that are still solved (i.e. the solvers returned *sat* or *unsat* as opposed to *unknown*). The timeout value was set to prevent unrealistic fuzzer runtimes for data collection. This bias towards timeout-inducing formulas reflects the imposed upper bound on solver runtimes, and the likelihood that, without such constraints, the true relative difference between solvers would be significantly larger than what is observed within a single generation. This fuels the discussion in Section 3.4 for the choice of $timeout$ for evaluation within the generational loop (five seconds) and for the final evaluation of the fittest formula (10 minutes).

3.2.2 SMT Solving

In order to evaluate the fitness function, I had to execute each SMT solver on each member of the population. This formed the *runtimes* list.

I chose to investigate Z3 (4.15.4), cvc5 (1.3.1) and MathSAT(5) as the SMT solvers, providing them with `QF_NIA` formulas.

3.2.2.1 Solver APIs

Python offers APIs for both Z3 and cvc5 – the two major SMT solvers. I first turned to these as a way of running my formula on the respective testers. However, I faced the problem of the differing syntax used by each API. While Z3 is tightly integrated with Python and supports operator overloading (allowing expressions such as $x > y$ to be written directly), cvc5 follows a different, more explicit model.

The cvc5 API represents variables (x, y) as *Terms* and expressions $(x > y)$ must be constructed using solver methods. There is no understanding of $Term1 > Term2$, unlike in the Z3 API. Instead, the correct formulation using the cvc5 API would be $solver.mkTerm(Kind.GT, x, y)$, after instantiating x and y each as a *Term*.

This core difference in support meant that this method would require using adapters and wrapper classes in order to represent the same fundamental SMT formula in the Z3 and cvc5 representations within the DEAP framework. I made some small attempts to do this, but quickly realised that any success was feeble as it provided no flexibility in terms of adding more theories, nor did it allow for any additional SMT solvers to be integrated into the fuzzer.

3.2.2.2 PySMT

I then looked to PySMT [27], a Python package which encapsulates Z3, cvc5 and a wide range of other solvers, allowing a single representation to be used as direct input to SMT solvers (rather than some intermediate representation which the API parses). This representation is SMT-LIB, the language of SMT solvers (see Appendix B for examples). This showed promise, but I discovered that PySMT does not support the `QF_NIA` for cvc5 (only for Z3).

3.2.2.3 Subprocesses

After attempts to utilise SMT-specific packages to aid my fuzzer, Python's subprocesses seemed the best alternative. By installing Z3, cvc5 and eventually MathSAT and adding them to my system environment, I could use subprocesses in order to invoke them directly from within the code. Again, the syntax for solver varied slightly but only on a parameter-level, not a structural or type-level. To utilise this, I had to convert each DEAP individual into SMT-LIB in order for the solvers to understand the SMT formula. The following snippet of code shows the process of converting a DEAP individual to its corresponding SMT-LIB formula, with the use of the Z3 API to formulate the assertions and convert them into SMT-LIB using $to_smt2()$:

```
# Compile DEAP expression into a Python representation
func = gp.compile(expr=individual, pset=pset)

# Create Z3 variables
vars_Z3 = [Int(f"x{i}") for i in range(NUM_VARS)]

# Build Z3 formula
Z3_formula = func(*vars_Z3)

# Use a Z3 solver to collect create the assertion
s = Solver()
s.add(Z3_formula)

# Manually set logic
logic_str = "(set-logic QF_NIA)\n"
```

```
# Convert solver with assertions to SMT-LIB string
smtlib_str = logic_str + s.to_smt2()
```

With this well-formed SMT-LIB, I could simply invoke a subprocess using the solver's name (e.g. "Z3"), which runs the executable for that solver and returns an instance of *subprocess.CompletedProcess*, which I here assign to *proc*:

```
if solver == "Z3":
    proc = subprocess.run([solver, "-in"], input=smtlib_str,
        text=True, stdout=subprocess.PIPE, stderr=subprocess.
        PIPE, timeout=timeout_seconds)

elif solver == "cvc5":
    proc = subprocess.run([solver, "--lang", "smtlib2"], input
        =smtlib_str, text=True,
    stdout=subprocess.PIPE, stderr=subprocess.PIPE, timeout=
        timeout_seconds)

elif solver == "mathsat":
    proc = subprocess.run([solver, "-input=smt2"], input=
        smtlib_str, text=True,
    stdout=subprocess.PIPE, stderr=subprocess.PIPE, timeout=
        timeout_seconds)
```

You can see in the code above that the exact syntax of the subprocess invocations for each SMT solver varies a little, as mentioned, but all return the same *CompletedProcess* output. This output, *proc*, stores various information about the subprocess such as *returncode* (0 if the process was a success), *stdout* (what the process printed, which in the case of SMT solvers is either `sat`, `unsat` or `unknown`), and *stderr* (stores any warnings or error messages).

I recorded the duration of the solve using Python's time package, reporting the runtime and output message back to the evolutionary loop. As described, this runtime contributes to the fitness value (relative runtime difference) for the individual, while the string output can be used to determine the status of the solve (timed out or not).

3.3 Evolutionary Loop

3.3.1 Mutation and Crossover

Mutation and crossover are a consistent feature of evolutionary algorithms. As documented, DEAP provided great access to built-in functions within my genetic fuzzer. It also enforces type safety by ensuring that the roots of the original and new subtrees share the same type, thus preventing variation from introducing syntax errors.

The following code defines the mutation:

```
    toolbox.register("expr_mut", gp.genFull, min_=0, max_=2)
    toolbox.register("mutate", gp.mutUniform, expr=toolbox.
        expr_mut, pset=pset)
```

The first line creates a function called `expr_mut` which generates a new random subtree up to a depth of 2 according to the defined primitive set (i.e. according to the same generation rules as presented in BNF in Section 3.1.2). The second line defines the actual mutation operator, `mutUniform()`. This selects a random node in the AST and replaces the subtree with the new tree generated by `expr_mut`. It can be thought of as a small member of the starting population. If the parent tree has a depth greater than 2, its root node cannot change. I imposed this limit to prevent mutation from becoming an effective reinitialisation of the population; with it, the formulas cannot be changed dramatically, which aligns more strongly with the nature of genetic algorithms.

I implement one small adjustment for crossover from DEAP's `cxOnePoint()`: the two individuals must be causing struggles for the same solver (a set intersection must exist) for crossover to take place. Each formula's *struggle solver(s)* are stored and updated throughout the evolutionary loop to enable this. The term struggle solver describes the solver (Z3, cvc5 or MathSAT) that takes significantly longer to solve the formula than the others. This is important as allowing arbitrary crossover between any formulas may have resulted in the performance-reducing constructs being mixed and lost, resulting in neither offspring remaining struggle-some.

In Section 2.5.1, I outline the criteria for valid mutation in a biological or algorithmic context:

1. Every individual in the population must be reachable by one or more valid mutations.
2. Mutations must not favour particular parts or directions in the potential population space.
3. Small mutations should be more probable than large ones.

With my fuzzer architecture, given a starting tree of depth ≤ 2, it is possible to reach every individual in the population with a series of mutations, satisfying the first criterion. The chosen mutation operator is completely random (achieving the second) and capped in depth (in order to achieve the third).

I utilised the *varAnd* method in DEAP, which applies both mutation and crossover automatically. While this eased implementation, I did have to clone my population before using *varAnd* as any crossover or mutation replaces parents. As a result, I cloned the population, applied mutation and crossover with varying probabilities and then removed any duplicates from instances where no mutation nor crossover occurred (a parent was not altered and therefore would occur twice in the population). Although duplication can help drive the average fitness of a population up, it undoubtedly leads to a lack of diversity and begins to work against the motivation for fuzzing in the first place.

3.3.2 Join

I introduced a custom variation operator "join". This operator simply concatenated two existing trees into a single, larger tree by placing an `and` node at the root to connect them. Given that both formulas are valid SMT assertions before being joined, their root

nodes are Boolean operators, therefore allowing a conjunction to be placed above them and maintaining syntactic validity. I did not want the `and` operator to be featured in the initial population, so I created a new primitive set (*pset_join*) that contained the exact primitives of the existing set, used for the generation of the initial population, but with the addition of the `and` primitive. Therefore, *pset_join* was the one considered in my *evaluate()* function to ensure all primitives, including `and`, would be recognised and compiled into the required SMT-LIB representation correctly. Ultimately, this allowed the *join* operator to introduce a new primitive at the root and for the resulting formula, with an increased depth, to be evaluated as usual.

I implemented join because my initial testing of SMT solvers demonstrated interesting behaviour when tasked with solving deeply nested formulas. I hypothesised that if we continued to build the depth of the population throughout the generation, we may not find a large differential in runtime (because every solver is struggling). Conversely, if this did not come to pass, it would expose weaknesses of a particular solver's ability to handle nesting.

3.3.3 Selection

For selection, I chose an elitism-based approach. My population size stays fixed throughout the generations of the fuzzer by reselecting the Top N fittest individuals at the end of each generation. This approach is strict and means that existing individuals, known as parents, are only replaced if a fundamentally fitter formula is produced via mutation, crossover or join (when used). Elitism refers to the design choice of automatically carrying over the best-performing individuals from one generation to the next without modification. My selection process is not elitist in the pure sense, as in theory every parent could be eliminated if enough variation took place, but with my chosen variation probability parameters (shown in Section 5.2), it is rare that N or more offspring are produced. Therefore, in the majority of cases, elitism exists and propagates the best existing formulas to the next generation.

3.4 Output

After the final generation, the highest fitness individual from the population is extracted and further analysed. Initially, due to the high computational cost of evolutionary algorithms, the timeout was limited to a few seconds (as a configurable parameter) to keep data collection practical. While this sped the fuzzer up significantly, it removed the ability of the fitness function to distinguish between population members with runtimes greater than the timeout; the fitness of formulas which cause a timeout is strictly proportional to the lower bound (which is the fastest solve time) because the upper bound is capped by the timeout. As a result, the formula with the highest fitness is rerun on the struggle solver identified during evolution. For this final evaluation, a timeout of 10 minutes is used. In the case that the fittest individual times out on two of the three solvers during the generational loop, it is rerun on both of the solvers and the longest of the two is considered. With the upper bound and the lower bound established, the final fitness (*relative difference*) is calculated.

Each fuzzer run records the relative difference of the highest fitness individual in a file along with the SMT formula itself, the struggle solver and the 10-minute timeout runtime. The seed is also noted.

Chapter 4

Use of Large Language Models

4.1 Strategy and Objective

To utilise LLMs as variation operators in place of the built-in DEAP counterparts, my prompt to the LLM had to consider multiple angles. Firstly, I told the LLM that it was a variation operator (specifying mutation or crossover in the actual prompt) with the aim of creating a formula that would take an SMT solver a long time to solve. I wanted the LLM operators to be more "direct" than the random DEAP functions (otherwise both fuzzers would be essentially random), but equally did not want to inform the LLM of my specific fitness function to ensure the creativity of the outputs was not compromised.

In Section 3.3.1, I show that DEAP mutation satisfies the criteria presented in Section 2.5.1. However, the stochasticity of LLMs makes enforcing the same requirements for my LLM-guided fuzzers difficult. Given the complexity of SMT formula, LLM mutation is limited to subtree mutation and therefore only partially satisfies the first requirement (as for a chosen formula, we cannot reach all possible individuals because root mutation is forbidden). This was a design choice, as allowing SMT formulas to have their root nodes changed is simply defining a new formula, nulling the concept of a mutation. Although this is allowed for formulas with a depth of two in the DEAP mutation, such shallow formulas have limited complexity and are generally easy for SMT solvers to solve, and therefore do not significantly impact the objective of stressing solver performance. As for the second and third requirements, there is no true guarantee that mutations do not favour particular directions in the population space, nor that the mutations are more probable to be small.

In fact, as LLMs generate tokens from a probability distribution learned during training, the choice of mutation is influenced primarily by patterns acquired during training, while the DEAP primitive tree structures in the prompt serve to condition and guide the form of the generated mutations. In theory, one could train a language model to purposefully skew its understanding of mutations in order to partially achieve the third requirement. However, my investigation into LLMs aimed to scrutinise the ability of state-of-the-art models to vary formulas in a less constrained way, not to determine the potential for them to align with criteria.

4.2 Representation and Format Selection

Next, the format for SMT representation had to be chosen. Using OpenAI's GPT-4o, I tested its ability to return a valid mutation in DEAP's internal representation (prefix notation with worded operators), Z3 Python (infix notation with mathematical symbols) and SMT-LIB format (prefix notation with logic setting, variable declaration and asserts) when given a valid SMT in the same format. Following 500 tests for each, there was little difference in their success rate, so I proceeded with the DEAP format, as parsing the LLM output back into the DEAP framework was easily done with an built-in function *from_string()*.

4.3 Prompt Engineering and Constraints

Having chosen the format to present my SMT formula in, I ensured that the LLM variation produced only valid arithmetic operators defined within my primitive set containing variables up to *NUM_VARS*, so as not to invalidate the parameter capping the number of variables. I did this by including my DEAP primitive set declarations, as well as the number of variables argument, in the prompt.

Finally, a "Rules" section in the prompt guided the response away from tendencies which diluted the output and made parsing more difficult in testing. This included instructing that only the mutated formula in DEAP representation should be output, omitting any markdown code fences. I also emphasised that common binary operators `AND`,`OR` and `NOT` were not permitted, as tests showed a tendency for their introduction. Finally, I noted that the root operator should not change.

The full prompts for LLM-guided mutation and crossover can be found in Appendix A.

4.4 Parsing and Error Handling

Despite these rules and their large success in guiding the LLM into useful responses, I had to handle cases where the output was malformed. For mutation, this was handled by simply attempting to parse the output back into a DEAP primitive tree object. Using a simple *try-catch* block, I either returned the correctly mutated individual or the parent individual in the case that the mutation was invalid. I did not check whether the output represented a strictly valid mutation (as performed by DEAP) because it was ultimately not important, and giving the LLM more freedom to alter a formula (even potentially against the instructions of the prompt) would not harm results – ensuring the output was a valid DEAP tree with the same root node sufficed for my use case. For crossover, however, which produces no "new" material, I had to constrain the output more to prevent the LLM-guided crossover from being too functionally dissimilar for meaningful comparison with the DEAP equivalent. The prompt itself was structured the same as for mutation, with the addition of introducing a `<SEP>` token to allow me to distinguish the end of the first formula and the start of the second. However, in parsing, after ensuring both individuals were syntactically valid, I checked their "make-up" of operators. I ensured that the number of each operator present in the two parents and

the two offspring was identical, including the root. Although this did not check that the constituent variables within each operator remained the same, testing proved that this was rarely misconfigured by the LLM.

4.5 Reliability and Success Metrics

To quantify the effectiveness of the LLM in generating syntactically valid variations, I recorded success statistics across three fuzzer runs of 30 generations each (90 generations in total).

LLM Variation	Successful	Total	Success Rate (%)
Mutation	845	947	89.23
Crossover	237	431	54.99

Table 4.1: Mutation and crossover success rate across 90 generations

Across 90 generations with a fixed population size of 25 and a mutation probability of 0.4, we would expect 900 mutations. Crossover is performed in pairs, leading to 12 attempts per generation and 432 crossovers in total (with the same probability). This justifies the *Total* column in Table 3.1, as we can observe the values similar to expected. We can see that mutation was largely successful with almost 90% of intended LLM mutations creating valid offspring. However, LLM crossover proved to be far less consistent, working just over half the time. Naturally, the prompt length for crossover is longer because two formulas are involved, along with the `<SEP>` token as opposed to a single formula for mutation. As a result, the attention mechanism (which relates different parts of the prompt to provide context) used in LLMs begins to diffuse with the struggles of tracking two lengthy SMTs. As a result, the offspring frequently contain at least one subtree which did not originate from either parent, which is flagged by my check, resulting in the original parents being returned with no change. Therefore, a combination of prompt length and output scrutiny resulted in LLM crossover being less common.

I considered increasing the crossover probability to achieve a variation rate closer to the desired 40%. However, this would mean that the parameter configuration for the LLM setup would differ from the baseline fuzzer, reducing their comparability. I deemed that the ability to consistently perform syntactically valid crossovers constitutes a meaningful requirement of an LLM-guided variation operator. Therefore, I maintained the crossover probability of 0.4 (and effective probability of approximately 0.22) throughout all experiments involving LLM-guided crossover.

Chapter 5

Results

5.1 Experimental Setup

My experiments were conducted using the following solvers: Z3 (4.15.4), cvc5 (1.3.1), and MathSAT(5). All SMT produced and given to these solvers were Quantifier-Free Non-Linear Arithmetic. The framework surrounding this was DEAP (1.4.3). A random seed was set and recorded for each fuzzer run to ensure reproducibility. Finally, all experiments were conducted on a personal laptop with the following specifications:

- Intel Core i5-1035G1 CPU @ 1.00GHz
- LPDDR4x RAM @ 3733MHz

5.2 Selection of Baseline Parameters

In order to quantify the impact of my LLM-guided variation and the "join" operator, while also attempting to classify the types of inputs that display a performance differential between solvers, my experiments had to be well-considered. I iterated through a list of values for each of the seven parameters, with a *default* value for each, which was chosen based on preliminary testing and intuition. The total number of values varied between three and five per parameter based on an initial testing round of three for each, and some further testing where results justified. Exhausting even the initial testing, with each fuzzer taking around an hour, would take $3^7 = 2187$ hours, which is equivalent to around 3 months. Even with efficient parallelisation, this would've been unachievable in my time frame. Therefore, I decided to only vary a singular parameter from the default value per run. This was also done in the hope of being able to clearly see the impact of the varied parameters on the results.

With five results for each varied parameter against the default, I could make justified decisions as to the chosen parameters for the baseline fuzzer. For population size and number of generations, the lower values gave poor results on numerous occasions simply due to not having enough time or population for evolution to be effective. For the number of variables, five produced the steadiest average and did not add an unwanted

Parameter	Parameter Values	Chosen Value
Population Size	10, 15, **20**, 25, 30	25
Number of Generations	10, 15, **20**, 25, 30	30
Number of Variables	3, **4**, 5	5
Maximum Initial Formula Depth	4, **5**, 6	5
Mutation Probability	0.0, 0.1, **0.2**, 0.3, 0.4	0.4
Crossover Probability	0.0, 0.2, 0.4, **0.6**, 0.8	0.4
Timeout (s)	1, **3**, 5, 10	5

Table 5.1: Parameter range for determining baseline values (default in bold)

constraint to the fuzzer (as a small number of variables do). Maximum depth was a strong parameter, and therefore, I maintained the default value of five. A maximum initialised depth of four was typically too easy for all solvers, while six caused too much immediate struggle. It is worth noting that this parameter determined only the maximum depth of the formulas at initialisation – the join operator is able to increase the depth beyond this value during evolution. For the mutation and crossover probabilities, I wanted to give the variation operators plenty of opportunity to impact the population, particularly as this baseline value was going to be used in the LLM fuzzer, which I aimed to quantify the utility of. Therefore, I chose 0.4 for both, and a value of 0.2 for the join probability (when used), as combining two formulas into one is a considerable alteration. Finally, for timeout, the value of three was the default and performed well. However, when trying a timeout of five seconds, the fuzzer results were consistently higher, at not too significant a runtime sacrifice.

Having selected my chosen set of baseline parameters, I ran the following fuzzer configurations 20 times each. The random seed was recorded to ensure reproducibility.

- Baseline Fuzzer (*Baseline*)
- Fuzzer with Join Operator (*Join*)
- Fuzzer with LLM-guided Mutation (*LLM Mut.*)
- Fuzzer with LLM-guided Crossover (*LLM Cx.*)
- Fuzzer with LLM-guided Mutation and Crossover (*LLM Mut.+Cx.*)

The following experimental evaluation aims to answer the following research questions: RQ1) Can genetic fuzzing find SMT formulas that expose performance differences between solvers? RQ2) Does enhancing the genetic fuzzer with LLM-based crossover and mutation improve the performance of the genetic fuzzer? RQ3) Are there common features of the SMT formula which trigger performance differences between solvers?

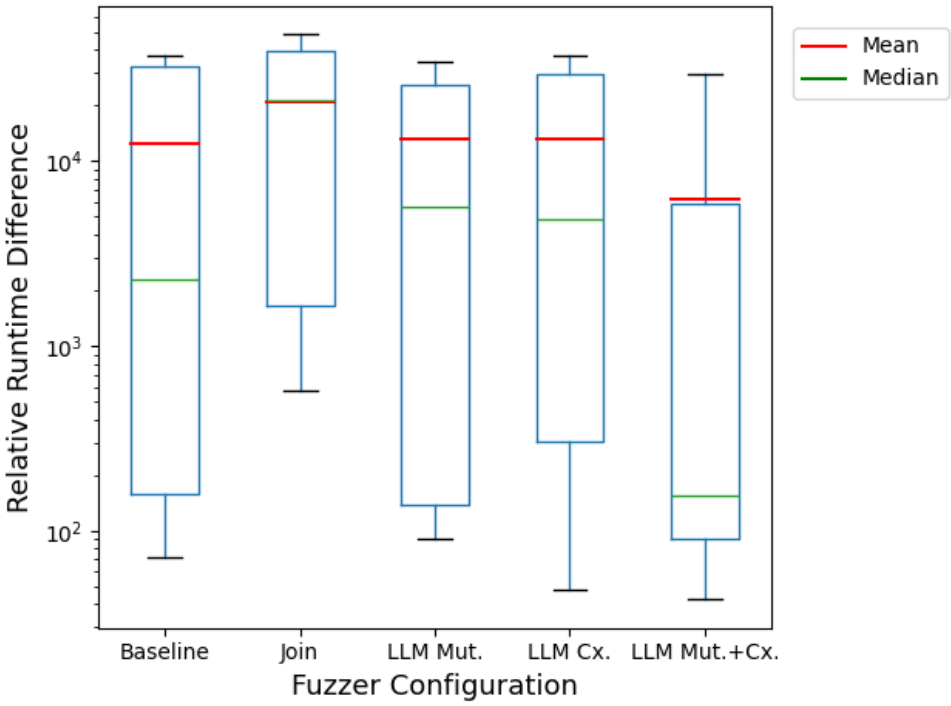


Figure 5.1: Distribution of greatest relative runtime difference for 20 runs

5.3 Performance of Genetic Fuzzing

5.3.1 Runtime Difference across Fuzzer Configurations

The set of box plots presented in Figure 5.1 provides an overview of the performance of the configurations. The minimum value across all 100 runs is 42.36, represented by the lower whisker of the `LLM Mut.+Cx.` configuration. With that being said, this still represents a formula that takes 43.36 (relative runtime difference plus one) times longer to be solved on, in this case, Z3, than the fastest between cvc5 and MathSAT. Referencing RQ1, it is clear that genetic fuzzing as an approach is consistently able to produce SMT formulas that are only solved efficiently by specific solvers.

The `join` configuration comfortably boasts the strongest results with a median relative runtime difference of over 10,000. This suggests that the introduction of the conjunction at the root, along with the subsequent depth increment, causes performance issues for the solvers. Further analysis into the impact of formula depth and the efficiency with which different solvers handle heavy nesting will be conducted in further sections.

5.3.2 Impact of LLM-Guided Variation Operators

The three LLM-guided configurations provide interesting results pertaining to the effects of multiple LLM-variation operators. The individual configurations, one using LLM-mutation (`LLM Mut.`) and the other using LLM-crossover (`LLM Cx.`), show similar results. By observing the mean relative performance difference, we can see that these two configurations perform similarly to the baseline configuration overall, demonstrating

the suitability of LLM-guided variation. Naturally, we would expect `LLM Mut.+Cx.` to perform at least as well as these two. However, the fifth box shows quite a significant performance drop off over the 20 runs. As documented in Section 4.5, GPT-4o was far from flawless, particularly in performing crossover, which raises the possibility that the compounded probability of LLM-failure resulted in a lack of variation, leading to inconsistent results.

For the `baseline` DEAP-operated configuration with mutation and crossover probabilities of 0.4, we have an effective $P(\text{change}_\text{B}) = 1-(1-0.4)(1-0.4) = 0.64$ for each formula to be mutated *or* recombined per generation. For `LLM Mut.`, we have an effective mutation rate of $(0.4 * 0.8923 = 0.3569)$, giving us a $P(\text{change}_\text{M}) = 0.614$. Moving left to right across the box plots, this value continues to reduce, with $P(\text{change}_\text{C}) = 0.532$, and finally $P(\text{change}_\text{MC}) = 0.498$. Therefore, although the `LLM Mut.+Cx.` configuration varies the given population less than half the time, we would expect a much bigger performance drop off between the LLM-mutation and LLM-crossover fuzzers (61.4% to 53.2%) if these statistics were to explain the far lower median relative runtime difference of the `LLM Mut.+Cx.` setup. However, for the `Baseline` (and `Join`) configurations, this assumes that every time the mutation and crossover probabilities are met, the variation is performed. For mutation, this is a valid assumption, as there are no additional conditions which need to be satisfied in order for mutation to take place. Crossover, as mentioned in Section 3.3, is only performed when the two formulas' struggle solver flags match. Therefore, if half the population struggle on cvc5 and the other half struggles on Z3, crossover would (on average) have its effective probability halved. If the entire population shared a single struggle solver, it would remain at 0.64.

This inconsistency, along with the considerable variance in the box plots, reveals an important feature of genetic fuzzing that will be prevalent throughout my analysis. Genetic fuzzing is inherently stochastic, meaning that performance can vary significantly between runs, irrespective of various parameters. In particular, the quality of the initial population (created randomly by DEAP's toolbox) plays a crucial role in shaping the fuzzing evolution, demonstrated by the similarity in performance between the top whisker of all five configurations (representing the strongest run out of the 20). The huge difference between mean and median for `LLM Mut.+Cx.` shows the runtime difference is heavily positively skewed, proving that the elite run (the upper whisker) was not a one-off occurrence and that multiple runs performed near this level despite there being enough poor runs to pull the median down.

Ultimately, despite the well-backed data on the ability of LLMs to produce mutations and recombinations, I cannot distinguish, with certainty, the performance of the baseline fuzzer and the LLM-guided counterparts. The 20 runs of each fuzzer configuration have shown that LLMs can absolutely compete with stochastic operators such as those in DEAP, but whether they are better or worse would require a different angle of approach from that I have taken. Therefore, I can say that RQ2 has been partially satisfied in that LLMs can be used as variation operators, but further investigation would be required to say whether they can be more effective than DEAP counterparts due to the high variance shown in the 20 runs.

5.4 Solver Struggle Distribution

Before attempting to identify aspects of SMT formulas that cause solver-specific performance degradation (RQ3), establishing which solvers struggle the most in general represents an important baseline before more focused and in-depth analysis can take place.

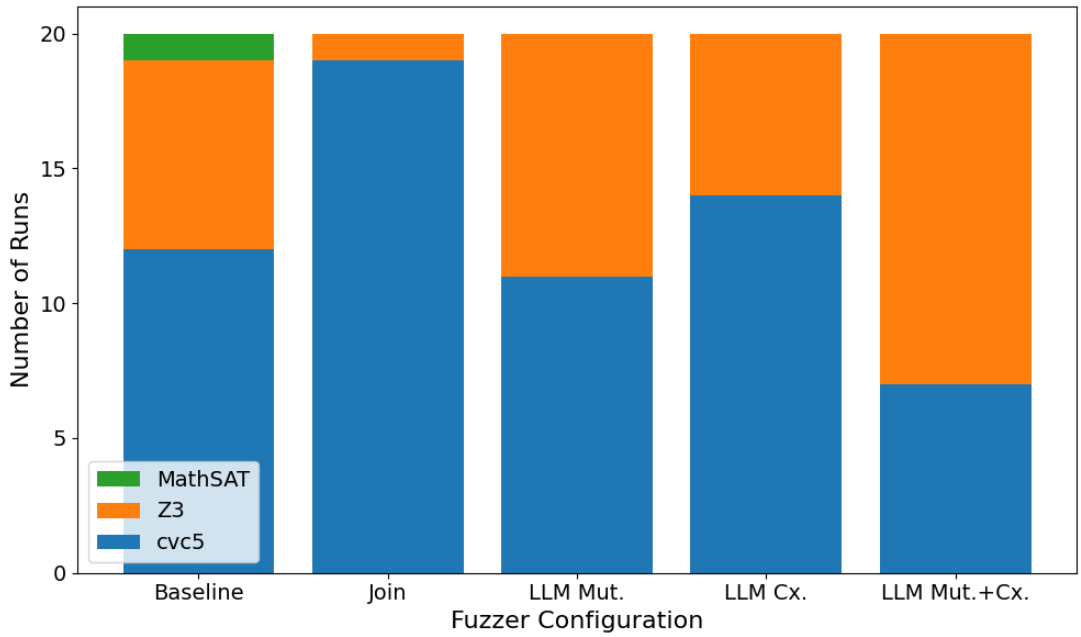


Figure 5.2: Distribution of struggle solver for 20 runs

Figure 5.2 presents interesting results as to the proportion of struggle solvers for the fittest formula for each configuration over the 20 runs. We can categorise the five configurations into two groups based on the contents of their populations (defined by the primitive set (`pset`) in DEAP). The baseline and all LLM-guided configurations span the same "formula-space", the set of all possible SMT that can be constructed given the primitives and operators available, in that any formula produced through the evolutionary process in one can also be produced in the other three. Join, however, as discussed in Section 3.3, introduces a Boolean `and` node as the root to join two existing SMT formulas and therefore can span a different formula-space in its population.

The stacked bar for `Join` has a significantly larger proportion of cvc5 struggles than the other configurations, which are more evenly split between Z3 and cvc5. This suggests that the difference in formula-space between the two groups of configurations is causing a significant runtime slowdown for cvc5. The concrete difference between them is the presence of the conjunction node at the root, though as `Join` also increases formula depth, it may be that the formula-space created by the join operator deepens the nesting within the population. In turn, this exposes cvc5's particular inefficiency in solving such formulas. This will be further discussed in sections to come.

5.5 Evolutionary Fitness Progression

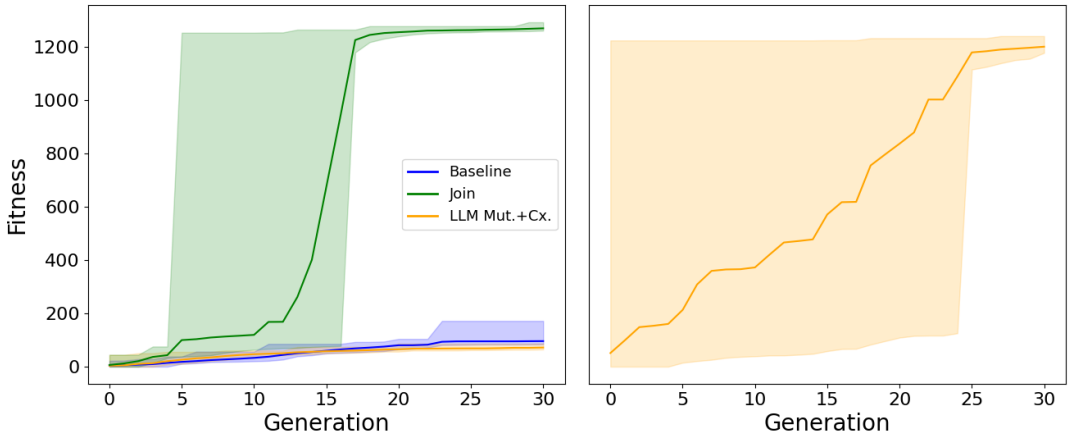


Figure 5.3: Fitness Range at each Generation for a single run

Figure 5.3 shows the minimum, average and maximum fitness amongst the population for a single run of the `Baseline`, `Join` and `LLM Mut.+Cx.` configurations (left) and a different, far more successful run of the `LLM Mut.+Cx.` fuzzer (right). The solid line represents the average fitness of the population at that generation, and the shaded area spans the minimum and maximum fitness values, covering the generation.

The left graph plots an average-performing `Join` run and subpar `Baseline` and `LLM Mut.+Cx.` runs. As a pair, the graphs further evidence the high variance shown by the box plots. One `LLM Mut.+Cx.` run can fail (maximum fitness/shaded area peaks at under 200) while another can be initialised with a formula that times out without any variation at all. The steep increase in fitness on the `Join` curve (compared to the right graph) can be explained by the difference in the presence of the join operator and the struggle solver tendencies observed in Figure 5.2. For the first, `Join` simply increases the amount of variation which takes place. With a probability of 0.2, the join operator increases total variation by 25% every generation (with mutation and crossover probabilities of 0.4) purely by existing as an additional variation operator. For the latter, and more interestingly, the fact that cvc5 dominates as the struggle solver increases the rate of variation further. The ratio of cvc5 to Z3 is 19:1, meaning that cvc5 exhibits the greatest relative runtime difference 95% of the time. As high-fitness formulas are propagated across generations, variation operators produce offspring that similarly challenge cvc5. Combined with `Join`'s tendency to target cvc5, the population quickly becomes dominated by cvc5-struggling formulas. Consequently, crossover becomes more frequent, as it only occurs between formulas sharing the same struggle solver.

This creates a loop whereby `Join` increases the production of formulas that struggle on cvc5. As these formulas propagate through the population due to their high fitness, the proportion of individuals whose struggle solver is cvc5 increases. Since crossover only occurs when both parents share the same struggle solver, this growing dominance of cvc5 raises the probability that randomly selected pairs are eligible for crossover. The

increased crossover rate in turn produces more offspring that inherit characteristics, or at least the essence, of these cvc5-struggling formulas, further reinforcing the solver imbalance in the population. Consequently, `Join` not only adds variation directly through its own operator but also indirectly amplifies variation by making crossover more likely. This positive feedback loop accelerates the search process, resulting in the sharper increase in fitness observed in the left graph.

5.6 Structural Causes of Solver Performance Differences

While the aforementioned loop explains `Join`'s success, there remains the question of whether the strong results stem from an increased depth or the introduction of the conjunction node at the root. This section will aim to answer this, along with whether or not Z3-dominated runs tend to produce lower relative runtime difference results.

5.6.1 Impact of Formula Depth on Solver Performance

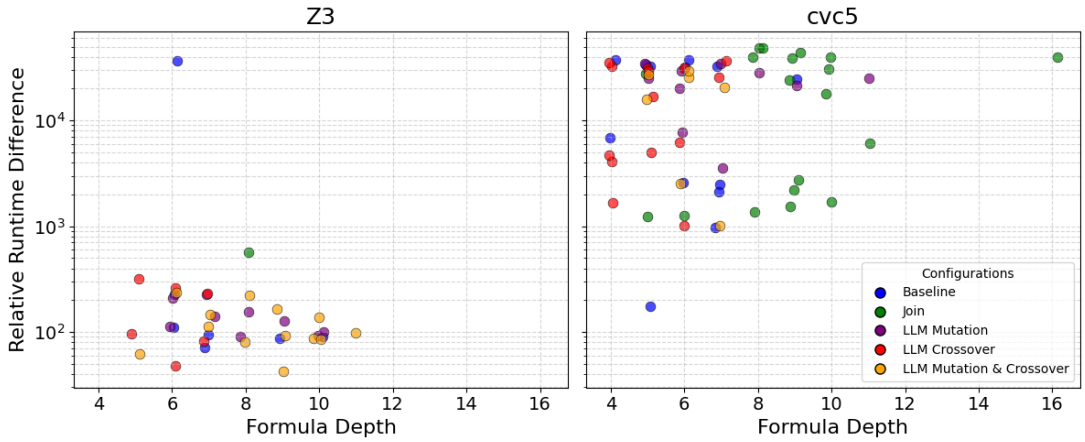


Figure 5.4: Depth against Relative Runtime Difference for every run (jittered)

Figure 5.4 speaks a very clear truth about the overarching performance levels of Z3 and cvc5. The average relative runtime difference for the cvc5-struggling formula is far higher than for Z3. In fact, ignoring the `Baseline` points in each graph which did not vertically cluster, a score of 700 acts as a perfect decision-boundary for classifying the struggle solver from the relative runtime difference. This explains the correlation between the proportion of cvc5 as the struggle solver (Figure 5.2) and the fuzzer performance (Figure 5.1). Despite this, Figure 5.4 shows that the relationship between depth and relative runtime difference is fairly weak. Naturally, we observe `Join` formula with large depth due to the mechanism of the operator. Although some of the `Join` formulas perform comparably to formulas from the remaining configurations, the elite performers score over 40,000, which is a level above the other configurations. We observe these formulas at a depth of 8 on the cvc5 graph, where formulas from multiple configurations reside on the Z3 graph with a much lower score. This shows that, while depth certainly impacts performance, it is not enough to account for the huge differential in solver performance.

We also observe that cvc5 is far more volatile than Z3. The stochasticity of fuzzing gives rise to variation in results, but it is far more significant in the cvc5 graph. All but two of the Z3-struggling formula score between 0 and 300, whereas all but one of the cvc5-struggling formula score between 900 and 40,000. This huge variation in results is also curtailed by the 10-minute timeout that was imposed on the final evaluation of the fittest formula; this range would be even bigger with no timeout. On the other hand, only a single Z3-struggle formula reached this timeout, showing that Z3's poor performance is more predictable in magnitude and rarely performs as badly, relatively, as cvc5 on SMT formulas. The likelihood for Z3 to solve a large, deeply-nested formula (of `QF_NIA`) is far higher than cvc5, making it a better choice for industry usage where such reliability is vital. Also, Figure 5.4 suggests that the introduction of the `and` node is responsible for `Join`'s results, rather than the increased depth.

5.6.2 Clustering Attempts

Depth is one of many extractable features of SMT formulas that could be considered for correlation analysis. However, single-feature analysis is an arduous process and relies on prior hypotheses. In this section, I attempt to extract and embed features that capture the wider context of the SMT formulas in the hope of discovering structure-based clusters that can be correlated to poor performance. If successful, this would allow a model to predict the extent of performance differential, as well as the struggle solver, purely from the SMT formula string. In addition, through clustering, I explore a hypothesis-driven approach on the impact of multiplication operators in Section 5.6.2.2.

5.6.2.1 Bag of Words

My first attempt at clustering these SMT formulas involved a standard Bag of Words (BOW) feature set. I extracted all parentheses, commas and operators from the SMT formulas (in DEAP representation) while ignoring variable names (ARGX). Although this does mean that $ARG0 * ARG0$ is interpreted the same as $ARG0 * ARG1$, I deem the decrease in feature set to be worth this loss of information. With this, each formula had a 13-dimensional vector representing the number of occurrences of each feature.

With these embeddings in place, I had to choose how they would be quantified in a two-dimensional setting to allow for graphing. I started with Principal Component Analysis (PCA), which reduces the number of dimensions in data to "principal" components (2) which retain most of the original information. This is done by transforming potentially correlated variables into a smaller set of variables (known as the principal components) [28]. The left subfigure, plotting the PCA reduction, had a variance of 65.23%, meaning that the two principal components capture just under two-thirds of the total variation in the data. This means that 65.23% of the data can be explained by these two dimensions alone. While this is a strong reduction, the graph shows that formula length plays a big role in this explained variance. We observe many `and`-rooted formulas, created by the `Join` fuzzer, separated from the main cluster. This aligns with the data shown in Figure 5.4 as many of the `and`-rooted formulas are deeply-nested. Naturally, a higher depth formula will contain more tokens in the BOW embedding. Besides this, no useful conclusions can be drawn from the Principal Component Analysis of the Bag of Words

feature set.

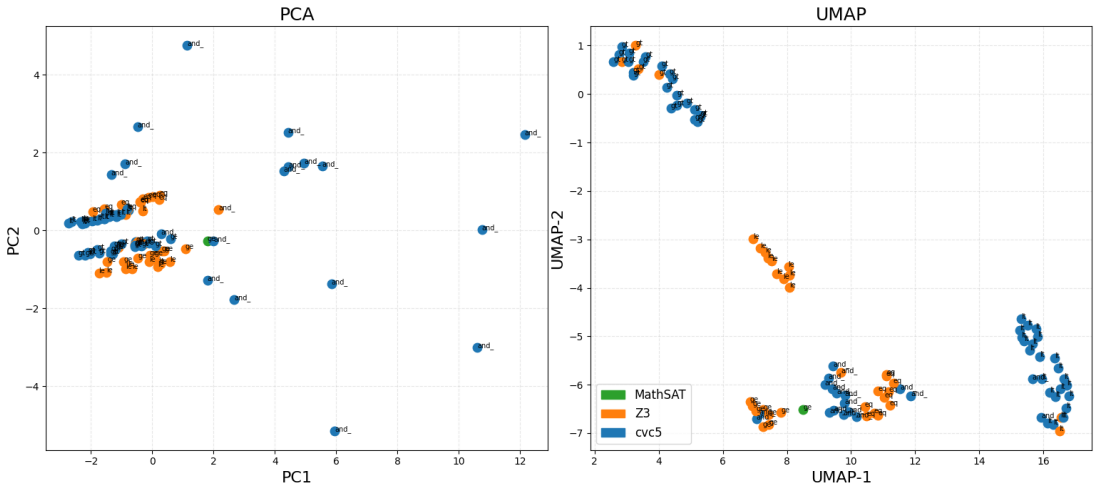


Figure 5.5: “Bag of Words” feature embeddings plotted using PCA and UMAP

My next dimension reduction technique was Uniform Manifold Approximation and Projection for Dimension Reduction (UMAP). UMAP represents high-dimensional data as a weighted graph, where edge weights reflect the likelihood of a connection between points. Each point is connected to its nearest neighbours, with the connection probability decreasing with distance. The number of neighbours each point connects to is determined by a parameter `n_neighbours`. The default value for standard use cases is 15. I chose to use 10 as my dataset is considerably smaller than what UMAP is typically used for. The low-dimensional graph is then optimised to be as structurally similar as possible to this high-dimensional representation, preserving the local and global structure of the original data [29]. Although UMAP’s intended use is on far larger and more complex datasets, the right subfigure of Table 5.5 shows clear clustering, including points with the same struggle solver. These clusters are of the root node. For example, the central Z3 cluster represents 10 formulas that all had a less than or equal to (*le*) node at their root. Given that the Bag of Words model contains the root node, it is logical that formulas sharing the same root node are considered in the same “neighbourhood” and UMAP’s projection surfaces this as visible clusters. This same clustering is not observed in the PCA projection, as the maximum variance in the feature vectors is dominated by formula length.

While the root node clustering doesn’t reveal anything interesting, the correlation with the struggle solver is significant. Table 5.2 formalises this.

We observe that cvc5 solves formulas with equality, less than or equal to and greater than or equal to root nodes quicker than Z3 and even MathSAT on occasion. Conversely, the less-than and greater-than nodes are dominated by MathSAT, with cvc5 struggling the most. cvc5 also struggles on the `and` formulas as discussed, though these contain a minimum of two Boolean subtrees (nested `and`s are possible) as arguments, so it is more difficult to conclude the cause for this. These results suggest that the method in which cvc5 solves strict and non-strict inequalities differs significantly, as relaxing the strict inequalities to include equality decreases solve time 30,000-fold. It is feasible that

Root	cvc5	Z3	MathSAT	Total
`and`	18	1	0	19
=	0	11	0	11
<=	0	10	0	10
<	23	2	0	25
>=	0	8	1	9
>	22	4	0	26
Total	63	36	1	100

Table 5.2: Root node counts by struggle solver.

cvc5 performs exceptionally well on equality SMT, such that the "or equal to" part of the inequality enables cvc5 to find a solution quickly. This will be further discussed in Section 5.6.3.

5.6.2.2 Hypothesis-Driven Features

Although the Bag of Words feature set revealed an interesting correlation between root node and struggle solver, I wanted to find purer structural properties of SMT formulas that could be correlated to a differential in performance across solvers (RQ3). In order to do this, I changed the feature set to focus more on specific properties of the SMT formulas. With a broad knowledge of the type of formulas that my fuzzers produced, I hypothesised that the presence of nested multiplications was a cause for performance issues. Therefore, my feature set consisted of three values: the ratio of multiply operators to total arithmetic operators, the mean depth of the arithmetic operators and the maximum depth of any arithmetic operator. I chose these features to avoid falling into the trap of depth-based clustering (as observed in the BOW PCA plot). Figure 5.6 shows the PCA and UMAP plots with the same configuration as the BOW feature set.

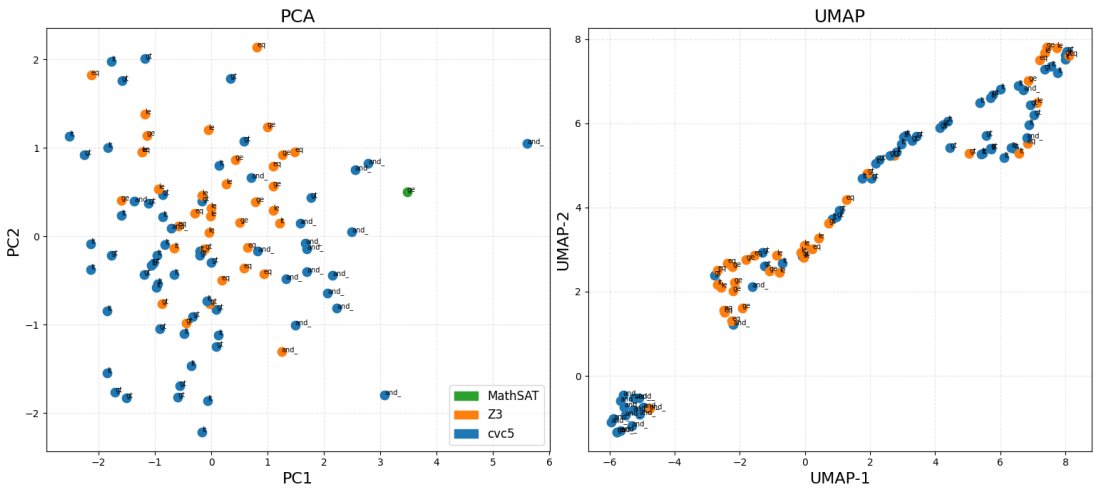


Figure 5.6: Operator feature embeddings plotted using PCA and UMAP

Despite a fresh feature set, Figure 5.6 reinforces the likelihood that clustering these SMT formulas with a more complex feature set in order to answer RQ3 is overly ambitious. The PCA plot does not provide any great insights beyond its visible similarity to the relative runtime difference against formula depth plot in Figure 5.4. This suggests that once again, even though the features are not count-based, the depth of the formula proves to be an axis of high variance that PCA cannot see past. Similarly, the UMAP plot appears to plot partial root-based clusters (notably the *and*-rooted formulas). Ultimately, a direct approach which plots simpler relationships (such as the number of multiplication operators against relative runtime difference) is required to determine the impact of particular features. This would remove the need for dimensionality reduction and allow for direct feature analysis. However, these features would have to be extracted first (which is what the attempted clustering aimed to prevent).

5.6.3 Formula Scrutinisation

Despite attempted clustering being largely unsuccessful, certain formulas within the wider 100 possess interesting traits. The following section looks more closely at four formulas, three of which have a depth of four, the lowest across all configurations, but still are unable to be solved within the 10-minute timeout. For all of these formulas cvc5 is the struggle solver. The fourth formula is the only one that identified MathSAT as the struggle solver. This section hopes to correlate structural features to solver-specific performance degradation (RQ3).

5.6.3.1 4-Depth Timeout Formula

The following formulas represent the lowest depth formulas in the population, in infix notation for readability, which are undecidable by cvc5 within 10-minutes of solving:

Formula 1: $\big((x_2+x_4)\cdot(x_0+x_4)+(x_4+x_0)-(x_0-x_2)\big) > \big((x_1+x_3)\cdot(x_3+x_0)\big)$

Formula 2: $\big((x_2\cdot x_4)\cdot(x_2-x_3)\cdot((x_2\cdot x_3)-(x_0\cdot x_3))\big) <$
$\big(((x_3+x_0)-(x_3\cdot x_0))\cdot((x_3-x_4)-x_3)\big)$

Formula 3: $\big((x_1\cdot x_2)\cdot(x_2\cdot x_3)\cdot((x_1-x_3)-(x_3+x_4))\big) <$
$\big(((x_2\cdot x_4)\cdot(x_1\cdot x_0))-((x_3\cdot x_0)\cdot(x_2\cdot x_2))\big)$

We can see that these formulas are rooted by either a greater-than or a less-than node, which we saw was a common occurrence for cvc5 struggles. These formulas all contain at least two levels of nested multiplication, with Formulas 2 and 3 containing three. Therefore, at a basic level, it seems that cvc5 tends to struggle much more than Z3 and MathSAT when dealing with this structure. However, as mentioned, this would require further feature-specific analysis to truly determine.

Given the tendencies shown in Table 5.2, I decided to rerun the three formulas on each solver with their non-strict (inclusive) bound equivalent. This revealed some staggering results across all formulas.

Formula	Root Node	cvc5	Z3	MathSAT
1	$>$	600	0.0274	0.0260
1	$>=$	0.0223	0.0510	0.0198
2	$<$	600	0.5021	0.0243
2	$<=$	0.0215	0.5244	0.0150
3	$<$	600	0.2807	0.0211
3	$<=$	0.0192	0.2518	0.0150

Table 5.3: Runtime (s) for strict and non-strict root node

Table 5.3 shows that by relaxing the root node to allow equality, cvc5 becomes able to solve the formulas faster than Z3 can. As alluded to earlier, cvc5 may be optimised towards solving equality formulas, and therefore the way it decomposes a $>=$ or $<=$ operator is oriented towards this strength. To investigate this, I convert such operators into their equivalent *or* (disjunction) forms. In SMT-LIB (prefix) form:

```
(>= x y) ≡ (or (= x y) (> x y))
```

For example, Formula 1 with a $>=$ root:

$$\big((x_2+x_4)\cdot(x_0+x_4)\cdot((x_4+x_0)-(x_0-x_2))\big) \geq \big((x_1+x_3)\cdot(x_3+x_0)\big) \equiv$$
$$\Big[\big((x_2+x_4)\cdot(x_0+x_4)+(x_4+x_0)-(x_0-x_2)\big) = \big((x_1+x_3)\cdot(x_3+x_0)\big)\Big]$$
$$\vee\Big[\big((x_2+x_4)\cdot(x_0+x_4)+(x_4+x_0)-(x_0-x_2)\big) > \big((x_1+x_3)\cdot(x_3+x_0)\big)\Big]$$

To determine cvc5's ability to solve the equality part of a non-strict inequality, I placed the equality branch first. If implemented, this would allow for short-circuit evaluation whereby cvc5 solves the equality part of the assertion and returns `sat` without consideration of the second operand. Along with the equivalent *or* and inequality formula, I evaluate the three formulas with an equality ($=$) root node in Table 5.4. Again, a 10-minute timeout was imposed here.

These experiments reveal curious cvc5 performance tendencies. Firstly, we observe that the formulas have very different runtimes despite representing the same SMT formula in logic. This shows that cvc5 does no node rewriting, at least between inclusive inequalities ($>=, <=$) and disjunctions, to increase performance. The equality results strongly suggest that cvc5 does not use short-circuit evaluation. If it did, we would expect the runtimes of the *or*-rooted and equality formula to be far closer (perhaps slightly longer for the disjunction, given the additional operator overhead). Table 5.4 does, however, provide strong results relating to the performance of non-strict bounds and equality. In all formulas, we observe that the runtime for the primary is slower by a few thousandths of a second, which is logical due to its additional complexity. It seems the methods for solving these two types of formula are closely interlinked, but without scrutinising cvc5's source code, I cannot make any concrete claims about this.

Formula	Root Node	cvc5 Runtime
1	$>=$	0.0223
1	*or*	76.63
2	$<=$	0.0215
2	*or*	149.6
3	$<=$	0.0192
3	*or*	600
1	$=$	0.0200
2	$=$	0.0169
3	$=$	0.0172

Table 5.4: Runtime (s) for the non-strict, disjunction and equality root node

In answering RQ3, these formulas have provided great insights. It is clear that the root node of a `QF_NIA` SMT formula is a key structure in predicting which solver will struggle. Furthermore, it has been shown that the way a given logical expression is represented by SMT can have a significant impact on not only solve runtime but even on solvability in some cases. While I scrutinise cvc5 here due to the low depth and high runtime of particular formulas, similar analysis could be conducted on Z3 and the reason for its struggles with non-strict inequalities.

5.6.3.2 The MathSAT Formula

Across the 100 fuzzer runs spanning five configurations, the greatest performance differential was observed in MathSAT only once. Firstly, it is worth noting that the relative runtime difference for this formula was 175.46, which represents a fairly weak fuzzer run when observing the median scores in Figure 5.1. This shows that MathSAT appears as the struggle solver only once because, in 99% of fuzzer runs, a greater performance differential was observed for another solver, meaning MathSAT was not the worst-performing solver in relative terms. While there may have been formulas in the population for which MathSAT was the struggle solver, it must not have performed *as* badly as other solvers on other formulas and consequently, the formula did not prevail as the highest fitness (elite) individual in that run.

The formula can be seen in Appendix B.1. Interestingly, it has the second-highest depth (13) across all elite formulas. The only deeper formula was an `and`-rooted formula (16) from the `Join` configuration, which was unsolved in 10 minutes by cvc5. While MathSAT performance may be strongly affected by formula depth, Figure 5.4 illustrates that across the 100 formulas, depth is not an accurate predictor of maximum performance differential. Features such as the root node and potentially others, which my clustering did not recognise, hold a stronger correlation.

Ultimately, without biasing the fuzzer towards MathSAT struggles and analysing these formulas, it is difficult to draw any conclusions about structural features that cause performance slowdown in MathSAT. It appears that, as the name suggests, it is well-suited to solving `QF_NIA` SMT formulas in general. MathSAT is shown to be the least

volatile of the three solvers, but not necessarily the solver with the lowest average runtime. For use cases where consistency matters more than raw speed, MathSAT may be a strong choice. In applications like formal verification or model checking [1], a solver that finishes within a predictable time is often more valuable than one that is slightly faster on average but occasionally times out.

5.6.4 Additional Findings

Throughout my formal and informal testing of SMT, I have encountered many interesting performance tendencies, the most relevant of which was the impact of root nodes and how equivalent forms may expose significantly different performance.

Additionally, and with less relevance to my research questions, I found an astounding bug in cvc5 independent of any logic whatsoever. For Formula 1 in Section 5.6.3.1, the order of variable declaration vastly affected solver performance. The full SMT-LIB formula can be found in Appendix B.2. The original order of declaration is as follows:

```
(declare-fun x4 () Int)
(declare-fun x0 () Int)
(declare-fun x1 () Int)
(declare-fun x2 () Int)
(declare-fun x3 () Int)
```

In this order, cvc5 is unable to solve the formula in the 10-minute timeout window. When rearranged in ascending order, cvc5 reports that the formula is `sat` in under 0.25 seconds. Conceptually, this is baffling; it is similar to how the order of independent variable declarations in a procedural program should not affect behaviour, yet in this case, it appears to influence performance. However, as explored in Section 1.1, these seemingly unexplainable features of SMT solvers are not uncommon. Rungta [2] discussed similar findings, stating that "renaming a variable" could result in inconsistency. Interestingly, attempting this same reordering on Formula 2, which contained declared variables $x3, x4, x0, x2$, I did not observe a cvc5 performance increase. This may be due to the absence of $x1$ in the formula, but ultimately, it is difficult to draw conclusions about this irregularity.

This discovery perfectly demonstrates the inconsistency and non-determinism that are central to ongoing SMT solver struggles. With purely syntactic problems like this, it is unsurprising that there exist structural features of SMT which expose weak performance in some solvers and not in others.

5.7 Limitations

5.7.1 Large Language Models

As documented in Section 4.5, my LLM-guided variation operators did not perform ideally, especially in the case of crossover (54.99% were syntactically valid). As a result, my effective variation probabilities were lower on the LLM-guided configurations in comparison to the `Baseline` (and `Join`) ones. Although my research question hoped

to determine the suitability of LLMs to act as variation operators, and the validity of outputs is part of that, the comparability of my results suffers as a result.

In order to judge this suitability better, more time would have to be spent testing different LLM models. Given my access to an OpenAI key through Edinburgh Language Models, an inexpensive and timely model was the focus of my model selection. After verifying that GPT-4o *could* perform the task after some basic prompt engineering, no further model analysis took place. Ideally, reliability and success metrics would be calculated for a variety of different models while exploring different prompts to maximise LLM-guided variation success. With this, the best model and prompt would be used in the LLM-guided genetic fuzzer to best imitate mutation and crossover operators.

An angle that went unexplored was using LLMs to generate initial populations. Although this wasn't a focus of my dissertation, it may have allowed for more effective analysis of LLMs' understanding of SMT and efficiency in imitating DEAP's genetic functions.

5.7.2 SMT Solving

Despite downloading the latest versions of cvc5, Z3 and MathSAT when research started, SMT solver updates are frequent, so there is no guarantee that the results I present in this dissertation will be reproducible now or in the future. However, the genetic fuzzer I developed (Algorithm 1) is agnostic to solver versions, and therefore these experiments could be run with updated solvers to generate new results.

In addition, a limitation of the fuzzer itself is the timeout (both the five seconds within the generational loop and the 10-minute final evaluation) implemented. Given the nature of the project, with software being changed as my understanding grew and the direction of results became clearer, my data collection was slightly constrained. Given that SMT solvers rely on CPU compute, all results had to be from a single machine to ensure solver runtimes were comparable across runs and configurations. Although compute servers were considered and may have allowed for longer timeouts, the extended data collection period and the need to test across multiple distinct configurations made them impractical for this study. Results were instead collected locally over time, which better suited the iterative nature of the software development.

5.7.3 Data Collection

High variance in the box plots provides evidence that my true understanding of fuzzer dynamics, including the LLM configurations, is limited. With more time, collecting additional data would help mitigate this issue. The random generation of the initial population meant the average fitness in some runs was significantly higher before any variation was even performed. Using a fixed starting population for all configurations over multiple runs would provide a better environment for comparison. However, this would come at the expense of genetic diversity and would hugely diminish my ability to analyse solver-specific correlations as required by RQ3. Therefore, this is a necessary trade-off between my research questions, but a limitation nonetheless.

Chapter 6

Conclusions

This dissertation investigated whether genetic fuzzing is an effective method for finding performance differences between SMT solvers and the role that LLMs can play in enhancing genetic operators to expose this. With these findings, the resulting SMT formulas were analysed with the aim of finding structural features that correlated with poor performance in particular solvers. To achieve this, an evolutionary framework was developed for a baseline and LLM-guided genetic fuzzer, allowing for the comparison of configurations.

The results show that genetic fuzzing is a very effective method for finding SMT solver performance issues, with formulas being solved up to 48,000 times slower on one solver than another in extreme cases. This is bounded by the imposed timeout, meaning the undecidability of some formulas spans beyond this and may even take hours, days or may never be solved. It was shown that the LLM-guided fuzzers could perform as well as the baseline, but without more runs, model testing and more focused experiments, it is not possible to say from my research whether they can enhance the evolutionary process at this time.

The main structural correlation found was the impact the root node of a formula can have on performance, with cvc5 struggling with [`and`, $<, >$] while Z3 performed less effectively on [$=, <=, >=$]. Additionally, it was shown that cvc5 lacks rewrite optimisations for some equivalent forms, short-circuit evaluation and stability surrounding arbitrary variable declaration.

Future Work There is potential for future work to be conducted on the absence of rewrite optimisations in cvc5 and other SMT solvers. Despite the plethora of optimisations that already exist within solvers, the ease with which such gaps were identified in this dissertation suggests that many more could be found. If reported to developers and implemented, this could significantly improve solver performance.

Additionally, LLM fine-tuning and prompt engineering in the context of SMT has yet to be formally explored. Further research in this area could improve the effectiveness of LLMs within evolutionary frameworks and even enable them to find structural correlations which expose weak performance themselves.

Bibliography

[1] George Cioçarlie, Cristian Cadar, et al. Fuzzing: Challenges and reflections. In *Proceedings of the ACM Symposium on Software Testing*, 2011.

[2] Neha Rungta. A billion smt queries a day. In *Verification, Model Checking, and Abstract Interpretation (VMCAI) / CAV 2022*, Lecture Notes in Computer Science. Springer, 2022.

[3] Yi Zhou et al. Mariposa: Detecting instabilities in smt solvers. In *IEEE Transactions on Software Engineering*, 2023.

[4] Yizheng Zhang, Clark Barrett, et al. Banditfuzz: Fuzzing smt solvers with reinforcement learning. In *Proceedings of the International Conference on Computer Aided Verification (CAV)*, 2020.

[5] Maolin Sun, Yibiao Yang, Yang Wang, Ming Wen, Haoxiang Jia, and Yuming Zhou. Smt solver validation empowered by large pre-trained language models. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1288–1300. IEEE/ACM, 2023.

[6] Maolin Sun, Yibiao Yang, and Yuming Zhou. Boosting skeleton-driven smt solver fuzzing by leveraging llm to produce formula generators. *arXiv preprint arXiv:2508.20340*, 2025.

[7] Xingyu Wu, Sheng-hao Wu, Jibin Wu, Liang Feng, and Kay Chen Tan. Evolutionary computation in the era of large language model: Survey and roadmap. *arXiv preprint arXiv:2401.10034*, 2024.

[8] Daniel Kroening and Ofer Strichman. *Decision Procedures: An Algorithmic Point of View*. Texts in Theoretical Computer Science. Springer, 2nd edition, 2016.

[9] Wikipedia. Satisfiability modulo theories. Accessed: 2025-10-10.

[10] de Engineer. Smt solvers. Accessed: 2025-10-10.

[11] Z3Prover. Z3 theorem prover, 2025.

[12] cvc5 Team. cvc5 smt solver, 2025.

[13] ACM. Fuzzing: Hack, art, and science. *Communications of the ACM*, 63(2), 2020.

[14] Barton P. Miller. Fall 1988 cs736 project list. Computer Sciences Department, University of Wisconsin-Madison, PDF, September 1988. Retrieved 2025-12-30.

[15] SMT-LIB Initiative. Smt-lib: The satisfiability modulo theories library, 2025. Accessed: 2025-10-10.

[16] Alain Pétrowski and Sana Ben-Hamida. *Evolutionary Algorithms*. John Wiley & Sons, 2017.

[17] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley Professional, Reading, MA, 1989.

[18] Xin Yao. Evolving artificial neural networks. *Proceedings of the IEEE*, 1999.

[19] F.A. Omara. Genetic algorithms for task scheduling problem. *Computers & Industrial Engineering*, 58(4):529–535, 2010.

[20] F.L. Custódio. A multiple minima genetic algorithm for protein structure prediction. *Computers in Biology and Medicine*, 52:1–8, 2014.

[21] Marek Obitko. Xi. crossover and mutation. Online, 2011. Retrieved 2011-04-07.

[22] A. E. Eiben and J. E. Smith. *Variation Operators (Mutation and Recombination)*, pages 31–32. Natural Computing Series. Springer, Berlin, Heidelberg, 2015.

[23] A. E. Eiben and J. E. Smith. *Representation, Mutation, and Recombination*, pages 49–78. Natural Computing Series. Springer, Berlin, Heidelberg, 2nd edition, 2015.

[24] Daniel Pétrowski and Sana Ben-Hamida. Adapting hardware systems by means of multi-objective evolution. *ResearchGate*, 2017.

[25] DEAP Developers. Deap: Distributed evolutionary algorithms in python. Accessed: 2025-10-10.

[26] Chuyang Chen, Brendan Dolan-Gavitt, and Zhiqiang Lin. Elfuzz: Efficient input generation via llm-driven synthesis over fuzzer space. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*, pages 6279–6298, Seattle, WA, USA, 2025. USENIX Association.

[27] pySMT Developers. pysmt: A solver-agnostic library for smt formula manipulation and solving, 2024.

[28] IBM. Principal component analysis (pca). *IBM Think*, 2024.

[29] PAIR Code. Understanding umap. *PAIR Code Interactive Article*, 2019.

Appendix A

LLM Operator Prompts

A.1 Crossover Prompt

```
You are an SMT crossover operator aiming to create HARD
   formula (formula that take SMT solver a long time to solve)

        Given these 2 DEAP PrimitiveTree individuals:

        {ind1}, {ind2}
        Here is the Primitive Set (pset) definition:

            # Arithmetic
            pset.addPrimitive(lambda x, y: x + y, [ArithRef,
               ArithRef], ArithRef, name="add")
            pset.addPrimitive(lambda x, y: x - y, [ArithRef,
               ArithRef], ArithRef, name="sub")
            pset.addPrimitive(lambda x, y: x * y, [ArithRef,
               ArithRef], ArithRef, name="mul")

            # Comparisons
            pset.addPrimitive(lambda x, y: x > y, [ArithRef,
               ArithRef], BoolRef, name="gt")
            pset.addPrimitive(lambda x, y: x < y, [ArithRef,
               ArithRef], BoolRef, name="lt")
            pset.addPrimitive(lambda x, y: x >= y, [ArithRef,
               ArithRef], BoolRef, name="ge")
            pset.addPrimitive(lambda x, y: x <= y, [ArithRef,
               ArithRef], BoolRef, name="le")
            pset.addPrimitive(lambda x, y: x == y, [ArithRef,
               ArithRef], BoolRef, name="eq")

            # Constants and terminals
            pset.addTerminal(BoolVal(True), BoolRef)
            pset.addTerminal(BoolVal(False), BoolRef)
```

```
Produce a crossover using only the 2 individuals
   provided by picking a node on each of the
   individuals and swapping the subtrees beneath that
   node.
Aim to make this crossover such that it will make the
   two resulting trees significantly harder for an SMT
    solver to solve.
Do NOT introduce any new material at all, simply
   perform the crossover (such that the summed lengths
    of the formula before and after are identical)
Choose this crossover point in a way such that the two
    resulting trees represent syntactically valid SMT
   formula (in DEAP format).

Rules:
- Output ONLY the formula, nothing else. and, or, nand
   , etc. are NOT permitted. Use only primitives
   provided in the set above
- The root operator remains a Boolean expression and
   the operands of this must evaluate to the same type
    to prevent type errors.
- Do NOT include any Markdown code fences like ```smt
   ``` or ``` or any other annotations.
- THIS IS IMPORTANT: Place a <SEP> token (formatted
   exactly like that) between the end of the first
   tree output and the start of the second so I can
   parse them easily.
```

Listing A.1: Prompt for the LLM-based crossover operator
```

A.2 Mutation Prompt

```
You are an SMT mutation operator aiming to create HARD
   formula (formula that take SMT solver a long time to
   solve)

Given this DEAP PrimitiveTree individual:

{ind}
Here is the Primitive Set (pset) definition:

    # Arithmetic
    pset.addPrimitive(lambda x, y: x + y, [ArithRef,
       ArithRef], ArithRef, name="add")
    pset.addPrimitive(lambda x, y: x - y, [ArithRef,
       ArithRef], ArithRef, name="sub")
    pset.addPrimitive(lambda x, y: x * y, [ArithRef,
       ArithRef], ArithRef, name="mul")

    # Comparisons
    pset.addPrimitive(lambda x, y: x > y, [ArithRef,
       ArithRef], BoolRef, name="gt")
    pset.addPrimitive(lambda x, y: x < y, [ArithRef,
       ArithRef], BoolRef, name="lt")
    pset.addPrimitive(lambda x, y: x >= y, [ArithRef,
       ArithRef], BoolRef, name="ge")
    pset.addPrimitive(lambda x, y: x <= y, [ArithRef,
       ArithRef], BoolRef, name="le")
    pset.addPrimitive(lambda x, y: x == y, [ArithRef,
       ArithRef], BoolRef, name="eq")

    # Constants and terminals
    pset.addTerminal(BoolVal(True), BoolRef)
    pset.addTerminal(BoolVal(False), BoolRef)

Produce a mutation using primitives defined in the pset,
   you may introduce new variables in the form of ARGx
   where x is a number up to {num_vars - 1}.
Aim to make this mutation such that it will make the
   expression significantly harder for an SMT solver to
   solve, but do NOT mutate the root operator and only
   mutate one side (subtree) of the root.
Do NOT use comparison operators (le, ge, eq, gt, lt) as
   children of other comparisons or arithmetic operators.
Output ONLY the mutated formula in DEAP format so I can
   parse it straight back into DEAP.

Rules:
- Output ONLY the formula, nothing else. and, or, nand,
   etc. are NOT permitted. Use only primitives provided in
```

```
        the set above
    - The root operator gives a Boolean expression and the
       operands of this must evaluate to the same type to
       prevent type errors.
    - Do NOT include any Markdown code fences like ```smt```
       or ``` or any other annotations.

    """
```

Listing A.2: Prompt for the LLM-based mutation operator

Appendix B

SMT-LIB Formula

B.1 MathSAT Struggle Formula

```
(set-logic QF_NIA)
; benchmark generated from python API
(set-info :status unknown)
(declare-fun x4 () Int)
(declare-fun x0 () Int)
(declare-fun x1 () Int)
(declare-fun x2 () Int)
(declare-fun x3 () Int)
(assert
 (let ((?x70 (* (+ (+ x0 x0) (+ (+ x2 x4) (* x2 x4))) (* (* x1 x0) x4))))
 (let ((?x76 (+ x0 x0)))
 (let ((?x77 (- x4 x0)))
 (let ((?x68 (- (- (+ x3 x0) (- x3 x4)) ?x77)))
 (let ((?x114 (* ?x68 (- (+ (* (* x2 x0) ?x76) x4) (+ x3 (* x1 x0))))))
 (let ((?x63 (* (* (* (+ (- x3 x4) (* ?x77 (+ x1 x3))) ?x114) x0) ?x76)))
 (let ((?x50 (* (+ (+ (+ x1 x1) (- x2 x4)) (* ?x77 (+ x1 x3)))
              (* ?x68 (- (+ ?x63 x4) ?x70)))))
 (let ((?x90 (* (+ (+ x2 x0) (* x1 x4)) (* (* x1 x0) (- x0 x4)))))
 (let ((?x58 (* (- (+ x2 x3) (+ x3 x4)) (+ (- x0 x3) (* x4 x0)))))
 (>= (* ?x58 ?x90) ?x50))))))))))
(check-sat)
```

B.2 Declaration Ordering Inconsistency Formula

```
(set-logic QF_NIA)
; benchmark generated from python API
(set-info :status unknown)
(declare-fun x4 () Int)
(declare-fun x0 () Int)
(declare-fun x1 () Int)
(declare-fun x2 () Int)
(declare-fun x3 () Int)
(assert
 (let ((?x70 (* (+ (+ x0 x0) (+ (+ x2 x4) (* x2 x4))) (* (* x1 x0) x4))))
 (let ((?x76 (+ x0 x0)))
 (let ((?x77 (- x4 x0)))
 (let ((?x68 (- (- (+ x3 x0) (- x3 x4)) ?x77)))
 (let ((?x114 (* ?x68 (- (+ (* (* x2 x0) ?x76) x4) (+ x3 (* x1 x0))))))
 (let ((?x63 (* (* (* (+ (- x3 x4) (* ?x77 (+ x1 x3))) ?x114) x0) ?x76)))
 (let ((?x50 (* (+ (+ (+ x1 x1) (- x2 x4)) (* ?x77 (+ x1 x3)))
               (* ?x68 (- (+ ?x63 x4) ?x70)))))
 (let ((?x90 (* (+ (+ x2 x0) (* x1 x4)) (* (* x1 x0) (- x0 x4)))))
 (let ((?x58 (* (- (+ x2 x3) (+ x3 x4)) (+ (- x0 x3) (* x4 x0)))))
 (>= (* ?x58 ?x90) ?x50))))))))))))
(check-sat)
```